

Chapter 56

Motor Control PWM (MCPWM)

56.1 Overview

The **Motor Control Pulse Width Modulator** (MCPWM) peripheral is intended for motor and power control. It provides six PWM outputs that can be set up to operate in several topologies. One common topology uses a pair of PWM outputs driving an H-bridge to control motor rotation speed and rotation direction.

The MCPWM can be divided into five main modules: PWM timers, PWM operators, Capture module, Event Task Matrix (ETM) module, and Fault Detection module. Each PWM timer provides timing references that can either run freely or be synced to other timers or external sources. Each PWM operator has all the necessary control resources to generate waveform pairs for one PWM channel. The Capture module is used for systems that need to accurately time external events. The ETM module responds to tasks received by the MCPWM, generating corresponding events depending on the state of motion. The Fault Detection module is used to capture external faults, allowing the system to respond by choice.

ESP32-P4 has two MCPWM peripherals, which are MCPWM0 and MCPWM1.

56.2 Features

An MCPWM peripheral has one clock divider (prescaler), three PWM timers, three PWM operators, a Capture module, an ETM module, and a Fault Detection module. MCPWM's core clock can be selected from three clock sources: PLL_F160M_CLK, XTAL_CLK, and RC_FAST_CLK (configured by the HP_SYS_CLKRST_MCPWM_n_CLK_SRC_SEL field of the [HP_SYS_CLKRST_PERI_CLK_CTRL20_REG](#) register). Figure 56.2-1 shows the submodules inside MCPWM and the signals on the interface. PWM timers are used for generating timing references. The PWM operators generate the desired waveform based on the timing references. Any PWM operator can be configured to use the timing references of any PWM timers. Different PWM operators can use the same PWM timer's timing reference to generate PWM signals, or different PWM timers' values to generate separate PWM signals. Different PWM timers can also be synchronized together.

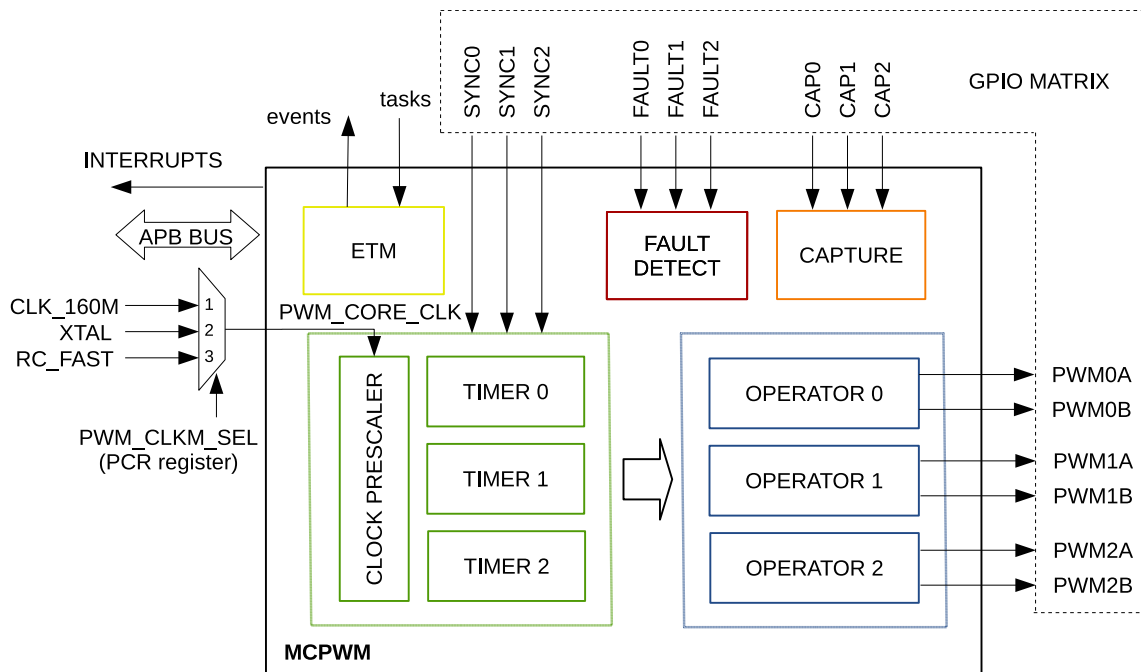


Figure 56.2-1. MCPWM Module Overview

Below is an overview of the submodules' functionality in Figure 56.2-1:

- PWM Timers 0, 1, and 2:
 - Every PWM timer has a dedicated 8-bit clock prescaler.
 - The 16-bit counter in the PWM timer can work in Count-Up Mode, Count-Down Mode, or Count-Up-Down Mode.
 - A hardware sync or software sync can trigger a reload on the PWM timer with a phase register. It will also trigger the prescaler's restart, so that the timer's clock can also be synced. The source of the hard sync can come from any GPIO or any other PWM timer's sync_out. The source of the soft sync comes from writing toggle value to the `MCPWM_TIMERn_SYNC_SW` bit.
- PWM Operators 0, 1, and 2:
 - Every PWM operator has two PWM outputs: PWMxA and PWMxB. They can work independently, in symmetric or asymmetric configurations.
 - The control of the PWM signal can be updated asynchronously.
 - Configurable dead time on rising and falling edges; each set up independently.
 - All events can trigger CPU interrupts.
 - Modulating of PWM output by high-frequency carrier signals, useful when gate drivers are insulated with a transformer.
 - Period, time stamps, and important control registers have shadow registers with flexible updating methods.
- Fault Detection Module:
 - Programmable fault handling in both cycle-by-cycle mode and one-shot mode.

- A fault condition can force the PWM output to either high or low logic levels.
- Capture Module:
 - Clock of the capture module is the same as MCPWM's core clock.
 - Speed measurement of rotating machinery.
 - Measurement of elapsed time between position sensor pulses
 - Period and duty cycle measurement of pulse train signals
 - Decoding current or voltage amplitude derived from duty-cycle-encoded signals of current/voltage sensors
 - Three individual capture channels, each of which with a time-stamp register (32-bit)
 - Selection of edge polarity and prescaling of input capture signals
 - The capture timer can sync with a PWM timer or external signals.
 - Interrupt on each of the three capture channels
- ETM Module:
 - Generation of different events depending on the different running states of each timer and operator.
 - Each timer and operator responds to its corresponding task and automatically performs the corresponding operation.
 - Each event and task can be enabled independently. When an event is not enabled, the corresponding event will not be generated. When a task is not enabled, the corresponding task will not be responded to.

56.3 Modules

56.3.1 Overview

The key modules in each MCPWM are timer module, operator module, fault detection module, capture module, and ETM module. See their functions in the following sections.

56.3.1.1 Timer Module

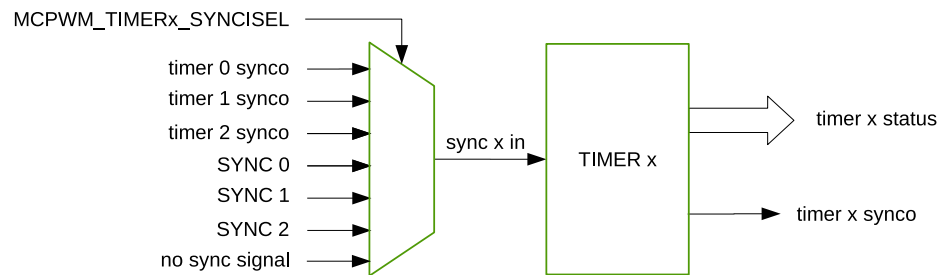


Figure 56.3-1. Timer Module

This module generates a timer, which can count at a specified period. It can work in Count-Up Mode, Count-Down Mode, or Count-Up-Down Mode. It supports different synchronization input sources (a total of seven optional input sources) for reloading the count value and direction, allowing synchronization among multiple timers. Furthermore, it provides synchronization output options (a total of four optional output sources) for use by other timers.

The timer x status in the figure represents the timer status output by the timer module, such as the counting mode, and count value equal to zero or period.

56.3.1.2 Operator Module

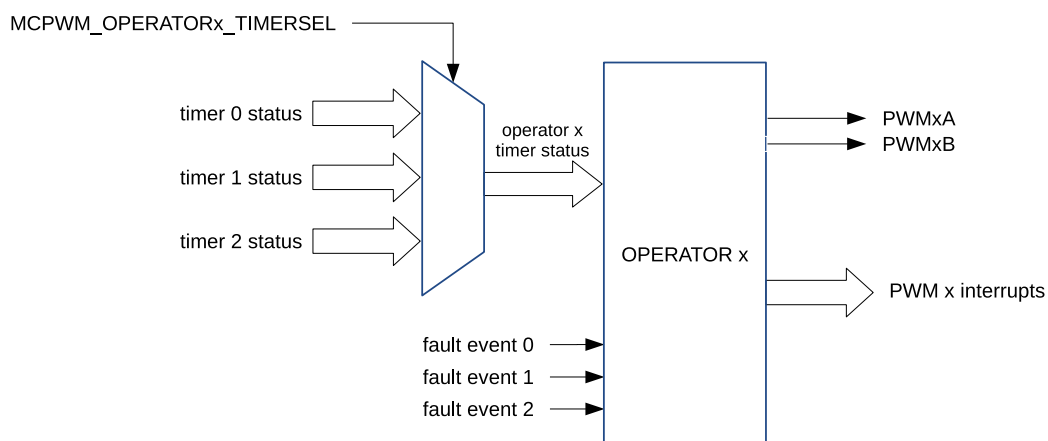


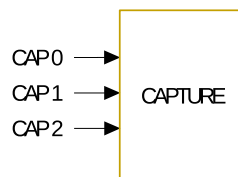
Figure 56.3-2. Operator Module

A PWM operator contains a PWM generator, a dead time generator, and a PWM carrier module. Their functions are shown in Table 56.3-1.

Table 56.3-1. Functions of Each Submodule in the Operator Module

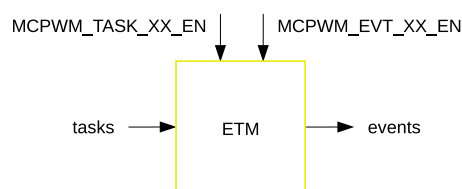
Submodule	Functions
PWM Generator	Generate PWM signals according to the configured duty cycle and waveform.
Dead Time Generator	Choose to add dead time to the rising and falling edges of the PWM signal generated by the PWM generator (add delays to the rising and falling edges), and perform operations such as inversion.
PWM Carrier	Choose to add a carrier to the PWM signal generated by the dead time generator. The carrier frequency and the first pulse duration of the PWM waveform after adding the carrier can be configured.
Fault Detector	Detect faults and generate corresponding fault events, respond to fault events based on the specified interval mode (one-shot mode or cycle-by-cycle mode), and when a fault event occurs, take action on the PWM signal generated by the PWM carrier in the specified way.

56.3.1.3 Capture Module

**Figure 56.3-3. Capture Module**

The module contains a timer that can be synchronized. When the input capture signal is valid, the count value of the timer is captured.

56.3.1.4 ETM Module

**Figure 56.3-4. ETM Module**

The module responds to received tasks, or generate and output events. Each event and task can be enabled independently.

56.3.2 PWM Timer Module

MCPWM has three PWM timer modules. Any of them can determine the necessary event timing for any of the three PWM operator modules. By using the synchronization signals from the GPIO matrix, built-in synchronization logic allows multiple PWM timer modules in one or more MCPWM peripherals to work together as a system.

56.3.2.1 Configurations of the PWM Timer Module

Users can configure the following functions of the PWM timer module:

- Control how often events occur by specifying the PWM timer frequency or period.
- Configure a particular PWM timer to synchronize phases with other PWM timers or modules.
- Configure the following timer counting modes: count-up, count-down, count-up-down.
- Change the rate of the PWM timer clock (PT_CLK) with a prescaler. Each timer has its own prescaler configured with `MCPWM_TIMER $n$ _PRESCALE` of the register `MCPWM_TIMER0_CFG0_REG`. The PWM timer increments or decrements at a slower pace, depending on the setting of this field. The new `MCPWM_TIMER $n$ _PRESCALE` configuration value will take effect when the timer stops and starts counting again.

56.3.2.2 PWM Timer's Working Modes and Timing Event Generation

The PWM timer has three working modes, selected by the PWM x timer mode field:

- Count-Up Mode:
The PWM timer increments from zero until reaching the value configured in the period field. Once done, the PWM timer returns to zero and starts increasing again. PWM period = the value of the period field + 1.
Note: The period field is `MCPWM_TIMER $n$ _PERIOD` ($x = 0, 1, 2$), i.e., `MCPWM_TIMER0_PERIOD`, `MCPWM_TIMER1_PERIOD`, `MCPWM_TIMER2_PERIOD`.
- Count-Down Mode:
The PWM timer decrements to zero, starting from the value configured in the period field. Once done, the PWM timer returns to the period value and starts decrementing again. In this case, the PWM period = the value of period field + 1.
- Count-Up-Down Mode:
This is a combination of the two modes mentioned above. The PWM timer starts increasing from zero until the period value is reached. Then, the timer decreases back to zero. The PWM timer cycles incrementally and decrementally in this mode. The PWM period = the value of the period field $\times$ 2.

Figures 56.3-5 to 56.3-8 show PWM timer waveforms in different modes, including timer behavior during synchronization events. In Count-Up mode, the counting direction after synchronization is always counting up. In Count-Down mode, the counting direction after synchronization is always counting down. In Count-Up-Down Mode, the counting direction after synchronization can be chosen by setting the `MCPWM_TIMER $n$ _PHASE_DIRECTION`.

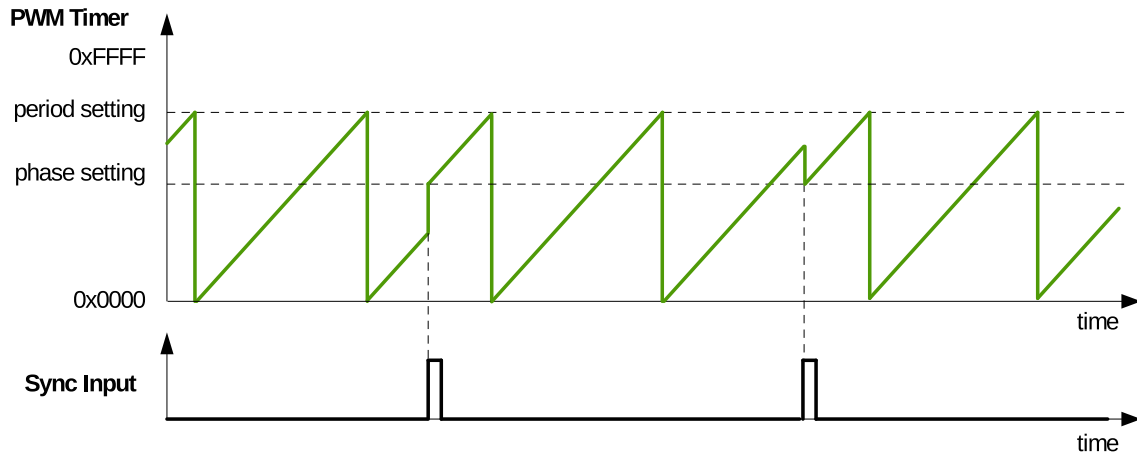


Figure 56.3-5. Count-Up Mode Waveform

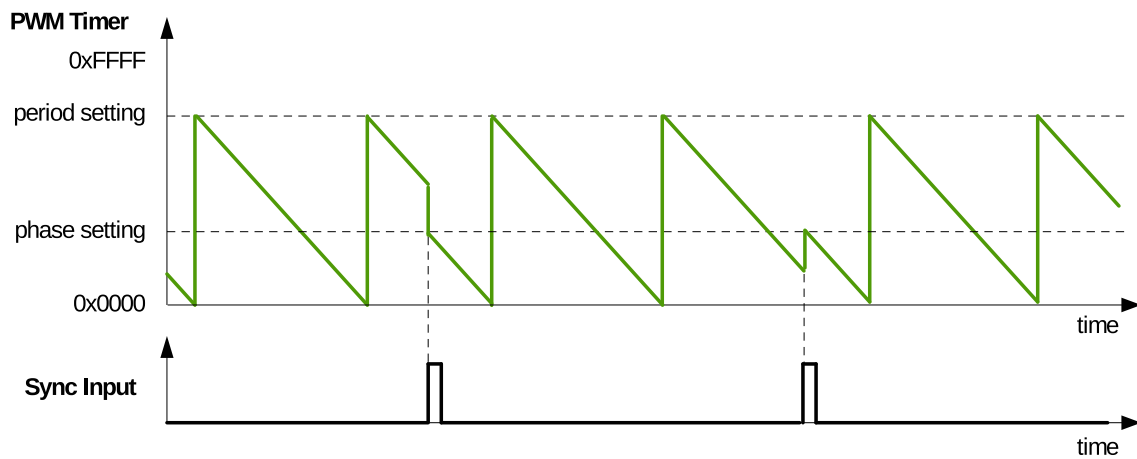


Figure 56.3-6. Count-Down Mode Waveforms

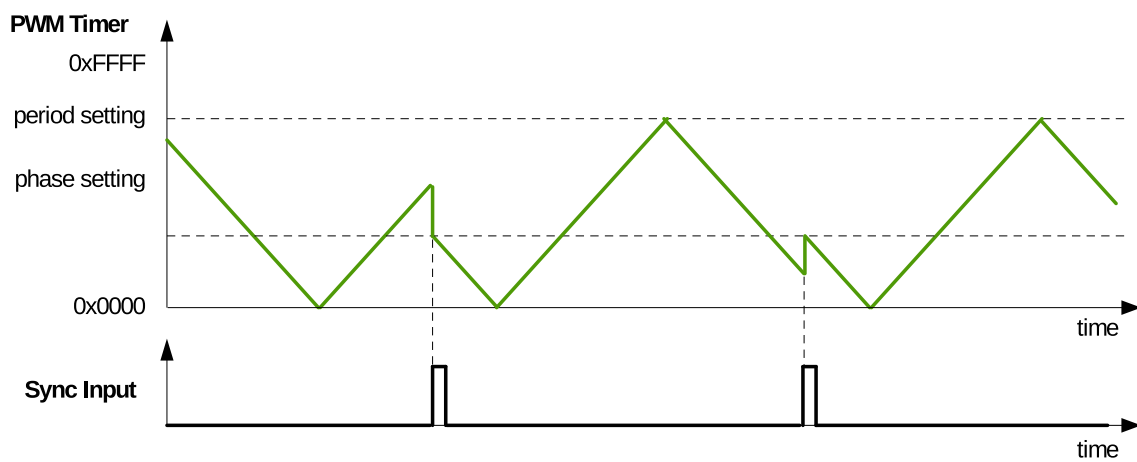


Figure 56.3-7. Count-Up-Down Mode Waveforms, Count-Down at Synchronization Event

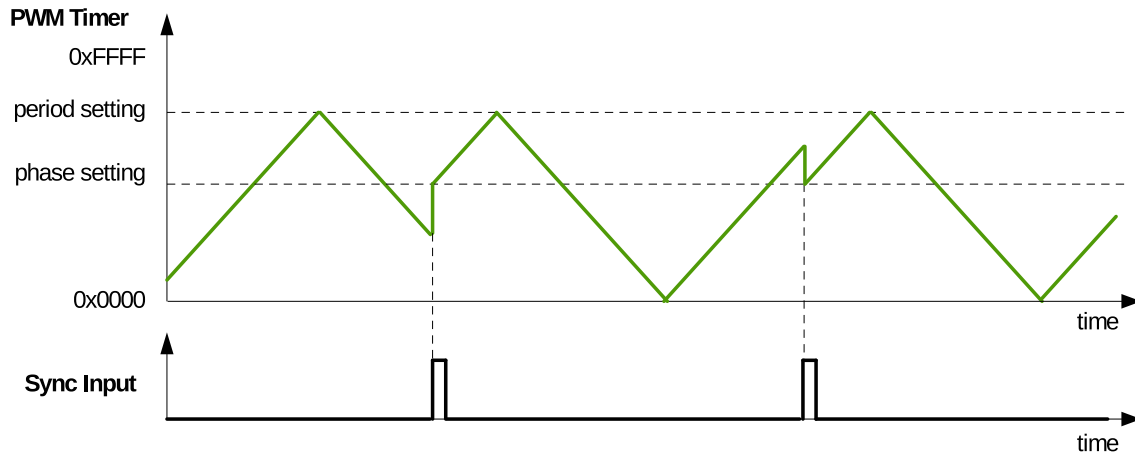


Figure 56.3-8. Count-Up-Down Mode Waveforms, Count-Up at Synchronization Event

When the PWM timer is running, it generates the following timing events periodically and automatically:

- UTEP: The timing event generated when the PWM timer's value is equal to the value of the period field (`MCPWM_TIMER $n$ _PERIOD`) and when the PWM timer is increasing.
- UTEZ: The timing event generated when the PWM timer's value equals zero and when the PWM timer is increasing.
- DTEP: The timing event generated when the PWM timer's value equals the value of the period field (`MCPWM_TIMER $n$ _PERIOD`) and when the PWM timer is decreasing.
- DTEZ: The timing event generated when the PWM timer's value equals zero and when the PWM timer is decreasing.

Figures 56.3-9 to 56.3-11 show the timing waveforms of U/DTEP and U/DTEZ.

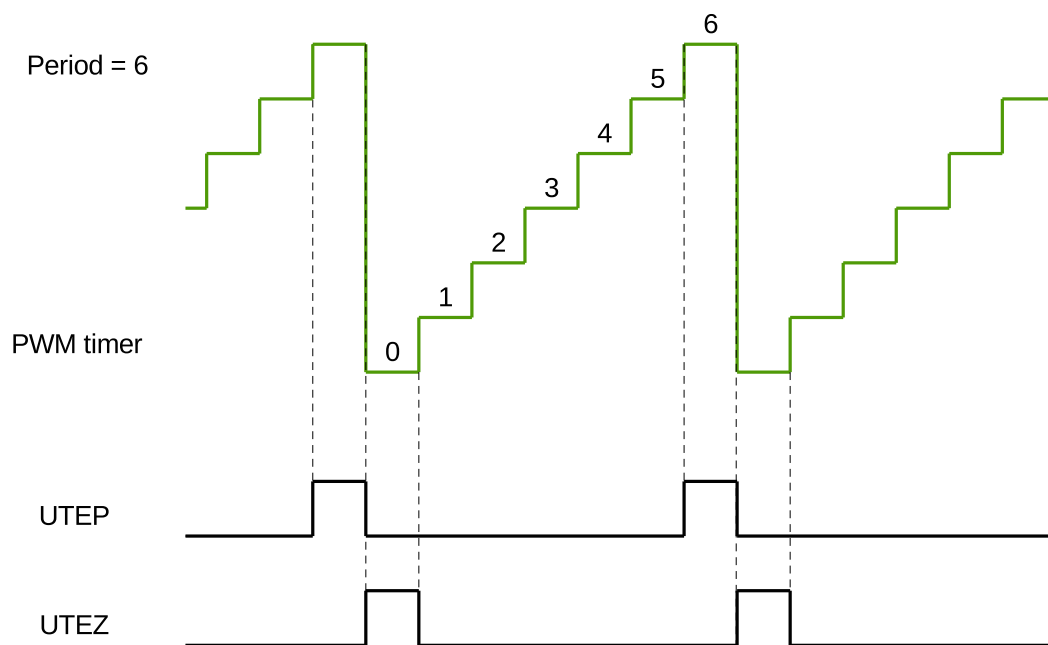


Figure 56.3-9. UTEP and UTEZ Generation in Count-Up Mode

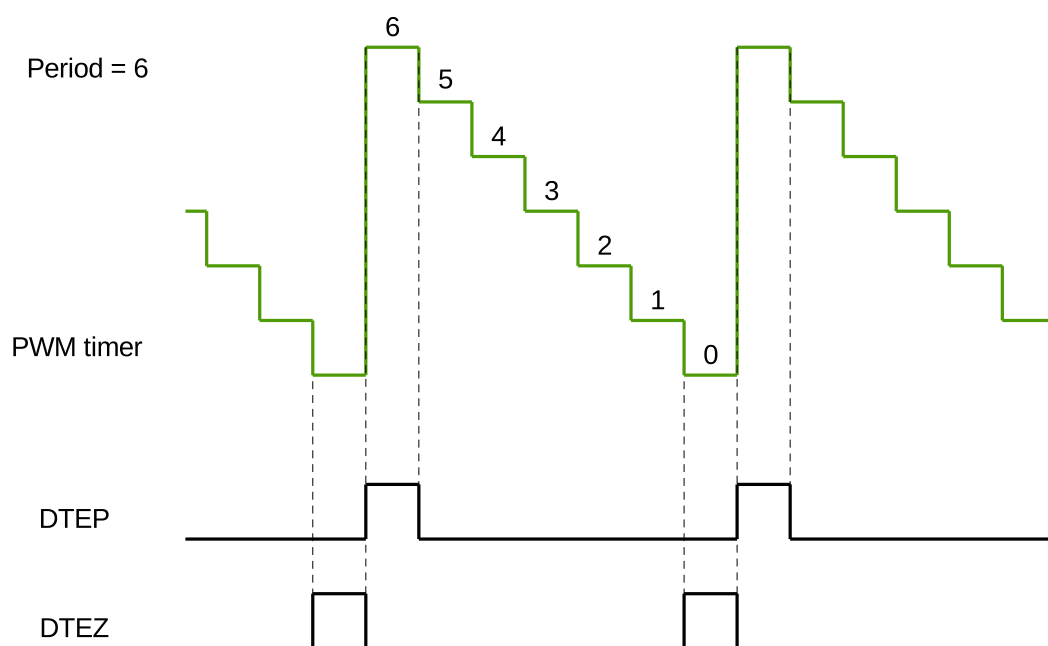


Figure 56.3-10. DTEP and DTEZ Generation in Count-Down Mode

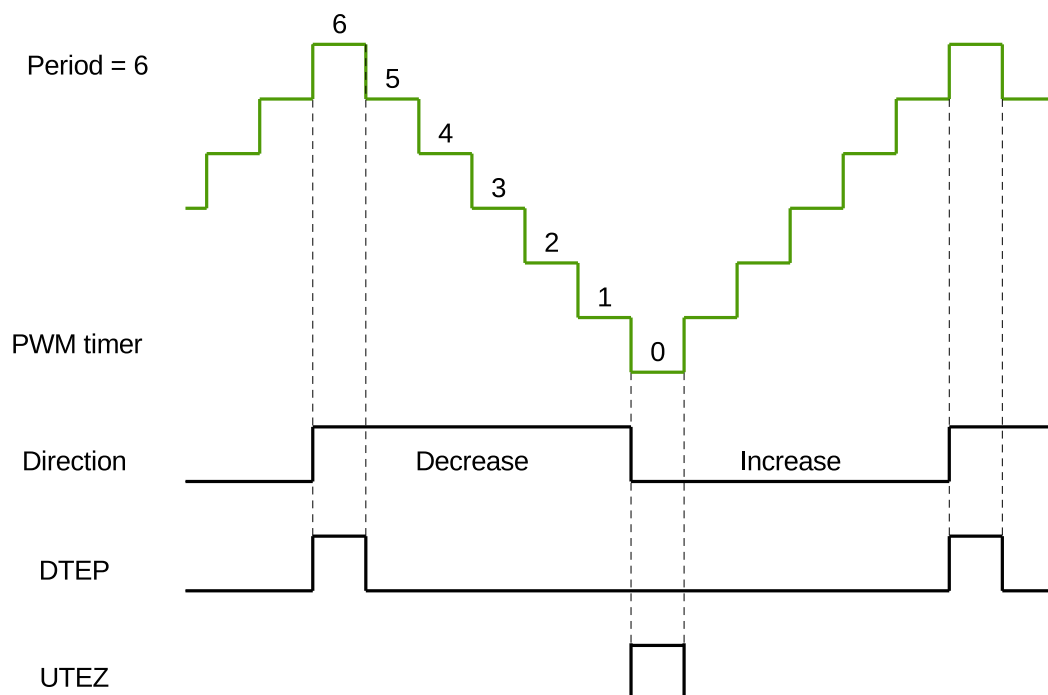


Figure 56.3-11. DTEP and UTEZ Generation in Count-Up-Down Mode

Please note that in the Count-Up-Down Mode, when the counting direction is increasing, the timer range is [0, period value - 1], and when the counting direction is decreasing, the timer range is [period value, 1]. That is, in this mode, when synchronizing the timer to 0, decreasing counting direction will be illegal, namely, `MCPWM_TIMER $n$ _PHASE_DIRECTION` cannot be set to 1. Similarly, when synchronizing the timer to period value, increasing counting direction will be illegal, namely, `MCPWM_TIMER $n$ _PHASE_DIRECTION` cannot be set to 0. Therefore, when the timer is synchronized to 0, the counting direction can only be increasing, and `MCPWM_TIMER $n$ _PHASE_DIRECTION` will be 0. When the timer is synchronized to the period value, the counting direction can only be decreasing, and `MCPWM_TIMER $n$ _PHASE_DIRECTION` will be 1.

56.3.2.3 Shadow Register of PWM Timer

The PWM timer's period register and the PWM timer's clock prescaler register have shadow registers. The shadow registers can back up the values that are about to be written to the valid registers. It also supports writing the values saved into the active register at a specific moment of hardware synchronization. The functionality of both register types is as follows:

- Active Register: Directly responsible for controlling all actions performed by hardware.
- Shadow Register: Acts as a temporary buffer for a value to be written to the active register. At a specific, user-configured point in time, the value saved in the shadow register is copied to the active register. Before this happens, the content of the shadow register has no direct effect on the controlled hardware. This helps to prevent erroneous operation of the hardware, which may happen when a register is asynchronously modified by software. Both the shadow register and the active register have the same memory address. The software always writes into, or reads from the shadow register.

The moment of updating the clock prescaler's active register is at the time when the timer starts operating. When `MCPWM_GLOBAL_UP_EN` is set to 1, the moment of updating the period active register can be selected by the following ways:

- By configuring the update method register `MCPWM_TIMERn_PERIOD_UPMETHOD` to 0, the update will start immediately.
- By configuring the update method register `MCPWM_TIMERn_PERIOD_UPMETHOD` to 1, the update can start when the PWM timer is equal to zero.
- By configuring the update method register `MCPWM_TIMERn_PERIOD_UPMETHOD` to 2, the update can start when the PWM timer is synchronized.
- By configuring the update method register `MCPWM_TIMERn_PERIOD_UPMETHOD` to 3, the update can start when the PWM timer is equal to zero or is synchronized.
- Software can also trigger a globally forced update bit `MCPWM_GLOBAL_FORCE_UP` which will prompt all registers in the module to be updated according to shadow registers.

56.3.2.4 PWM Timer Synchronization and Phase Locking

The PWM modules adopt a flexible synchronization method. Each PWM timer has a synchronization input and a synchronization output. The synchronization input can be selected from three synchronization outputs of the three PWM timers and three synchronization signals from the GPIO matrix. The synchronization output can be generated when the PWM timer's value equals to period or zero, the synchronization input signal is active, or software synchronizes. Thus, each PWM timer can flexibly select the synchronization input for different phase synchronization methods. During synchronization, the PWM timer clock prescaler will reset its counter and restart dividing the clock in order to ensure the correctness of the timer.

56.3.3 PWM Operator Module

The PWM Operator module has the following functions:

- Generates a PWM signal pair, based on timing references obtained from the corresponding PWM timer.

- Each signal out of the PWM signal pair includes a specific pattern of dead time (i.e., adding delays to the rising and falling edges of the PWM signal).
- Superimposes a carrier on the PWM signal, if configured to do so.
- Handles response under fault conditions.

Figure 56.3-12 shows the block diagram of a PWM operator.

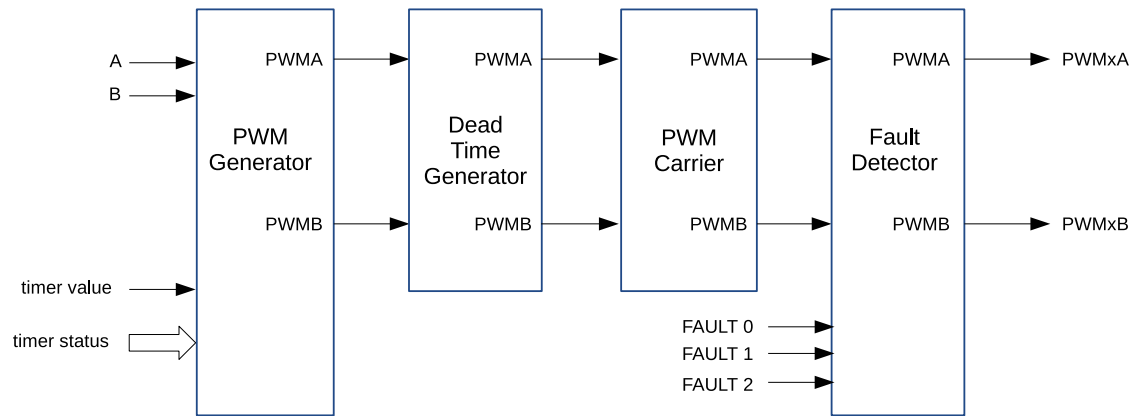


Figure 56.3-12. Block Diagram of A PWM Operator

56.3.3.1 PWM Generator Module

Purpose of the PWM Generator Module

In this module, important timing events are generated or imported. The events are then converted into specific actions to generate the desired waveforms at the PWMxA and PWMxB outputs.

The PWM generator module performs the following actions:

- Generation of timing events based on time stamps configured using the A and B registers. Events happen when the following conditions are met (for the configuration of registers A and B, see section [56.3.3.1](#)):
 - UTEA: the PWM timer is counting up and its value is equal to register A.
 - UTEB: the PWM timer is counting up and its value is equal to register B.
 - DTEA: the PWM timer is counting down and its value is equal to register A.
 - DTEB: the PWM timer is counting down and its value is equal to register B.
- Generation of U/DTO, U/DT1 timing events based on fault or synchronization events.
 - UTO: the PWM timer is counting up and FAULT0 detected (field [MCPWM_GENn_TO_SEL](#) is set to 0) or FAULT1 detected (field [MCPWM_GENn_TO_SEL](#) is set to 1) or FAULT2 detected (field [MCPWM_GENn_TO_SEL](#) is set to 2) or synchronized (field [MCPWM_GENn_TO_SEL](#) is set to 3).
 - UT1: the PWM timer is counting up and FAULT0 detected (field [MCPWM_GENn_T1_SEL](#) is set to 0) or FAULT1 detected (field [MCPWM_GENn_T1_SEL](#) is set to 1) or FAULT2 detected (field [MCPWM_GENn_T1_SEL](#) is set to 2) or synchronized (field [MCPWM_GENn_T1_SEL](#) is set to 3).
 - DTO: the PWM timer is counting down and FAULT0 detected (field [MCPWM_GENn_TO_SEL](#) is set to 0) or FAULT1 detected (field [MCPWM_GENn_TO_SEL](#) is set to 1) or FAULT2 detected (field [MCPWM_GENn_TO_SEL](#) is set to 2) or synchronized (field [MCPWM_GENn_TO_SEL](#) is set to 3).
 - DT1: the PWM timer is counting down and FAULT0 detected (field [MCPWM_GENn_T1_SEL](#) is set to 0) or FAULT1 detected (field [MCPWM_GENn_T1_SEL](#) is set to 1) or FAULT2 detected (field [MCPWM_GENn_T1_SEL](#) is set to 2) or synchronized (field [MCPWM_GENn_T1_SEL](#) is set to 3).
- Management of priority when these timing events occur concurrently.
- Generation of set, clear, and toggle actions, based on the timing events.
- Controlling of the PWM duty cycle, depending on the configuration of the PWM generator module.
- Handling of new time stamp values, using shadow registers to prevent glitches in the PWM cycle.

Shadow Register of PWM Generator

The time stamp registers A and B used by the hardware have shadow registers, which are registers [MCPWM_GENn_A_REG](#) and [MCPWM_GENn_B_REG](#). Shadowing provides a way of updating registers in sync with the hardware.

When [MCPWM_GLOBAL_UP_EN](#) is set to 1, the shadow registers can be written to the active register at a specified time. The update method field for [MCPWM_GENn_A_REG](#) and [MCPWM_GENn_B_REG](#) is [MCPWM_GENn_CFG_UPMETHOD](#). Software can also trigger a globally forced update bit

[MCPWM_GLOBAL_FORCE_UP](#) which will prompt all registers in the module to be updated according to shadow registers. For a description of the shadow registers, please see Section [56.3.2.3](#).

Timing Events

For convenience, all timing signals and events are summarized in Table [56.3-2](#).

Table 56.3-2. Timing Events Used in PWM Generator

Signal	Event Description	PWM Timer Operation
DTEP	PWM timer value is equal to the period register value	PWM timer counts down
DTEZ	PWM timer value is equal to zero	
DTEA	PWM timer value is equal to register A	
DTEB	PWM timer value is equal to register B	
DTO event	Based on fault or synchronization events	
DT1 event	Based on fault or synchronization events	
UTEP	PWM timer value is equal to the period register value	PWM timer counts up
UTEZ	PWM timer value is equal to zero	
UTEA	PWM timer value is equal to register A	
UTEB	PWM timer value is equal to register B	
UTO event	Based on fault or synchronization events	
UT1 event	Based on fault or synchronization events	
Software-force event	Software-initiated asynchronous event	N/A

The purpose of a software-force event is to impose non-continuous or continuous changes on the PWM_xA and PWM_xB outputs. The change is done asynchronously. Software-force control is handled by the [MCPWM_GEN_n_FORCE_REG](#) registers.

The selection and configuration of T0/T1 in the PWM generator module is independent of the configuration of fault events in the fault handler module. A particular trip event may or may not be configured to cause trip action in the fault handler submodule, but the same event can be used by the PWM generator to trigger T0/T1 for controlling PWM waveforms.

It is important to know that when the PWM timer is in Count-Up-Down Mode. It will always decrement after a TEP event, and increment after a TEZ event. So, when the PWM timer is in Count-Up-Down Mode, DTEP and UTEZ events will occur, while UTEP and DTEZ events never occurs.

The PWM generator can handle multiple events at the same time. Events are prioritized by the hardware and relevant details are provided in Table [56.3-3](#) and Table [56.3-4](#). Priority levels range from 1 (the highest) to 7 (the lowest). Please note that the priority of TEP and TEZ events depends on the PWM timer's counting mode.

If the value of A or B is set to be greater than the period, then U/DTEA and U/DTEB will never occur.

Table 56.3-3. Timing Events Priority When PWM Timer Increments

Priority Level	Event
1 (highest)	Software-forced event
2	UTEP

Priority Level	Event
3	UTO
4	UT1
5	UTEB
6	UTEA
7 (lowest)	UTEZ

Table 56.3-4. Timing Events Priority when PWM Timer Decrements

Priority level	Event
1 (highest)	Software-forced event
2	DTEZ
3	DT0
4	DT1
5	DTEB
6	DTEA
7 (lowest)	DTEP

Notes:

1. UTEP and UTEZ do not happen simultaneously. When the PWM timer is in Count-Up Mode, UTEP will always happen one cycle earlier than UTEZ, as demonstrated in Figure 56.3-9, so their action on PWM signals will not interrupt each other. When the PWM timer is in Count-Up-Down Mode, UTEP will not occur.
2. DTEP and DTEZ do not happen simultaneously. When the PWM timer is in Count-Down Mode, DTEZ will always happen one cycle earlier than DTEP, as demonstrated in Figure 56.3-10, so their action on PWM signals will not interrupt each other. When the PWM timer is in Count-Up-Down Mode, DTEZ will not occur.

PWM Signal Generation

The PWM generator module controls the behavior of outputting PWMxA and PWMxB when a particular timing event occurs. The timing events are further qualified by the PWM timer's counting mode (increment or decrement). Knowing the counting mode, the module may then perform an independent action at each stage of the PWM timer counting up or down.

The following actions may be configured on PWMxA and PWMxB outputs:

- Set High: Set the output of PWMxA or PWMxB to a high level.
- Clear Low: Clear the output of PWMxA or PWMxB by setting it to a low level.
- Toggle: Change the current output level of PWMxA or PWMxB to the opposite value. If it is currently pulled up, then pull it down, or vice versa.
- Do Nothing: Keep both outputs PWMxA and PWMxB unchanged. In this state, interrupts can still be triggered.

Actions on outputs is configured by using registers `MCPWM_GENn_A_REG` and

[MCPWM_GENn_B_REG](#). So, the action to be taken on each output is set independently. Also, there is great flexibility in selecting actions to be taken on a given output based on events. More specifically, any event listed in [Table 56.3-2](#) can operate on either output of PWMxA or PWMxB. To check out registers for particular generator 0, 1, or 2, please refer to the register description in [Section 56.5](#).

Waveforms for Common Configurations

In the configuration of various waveforms below, period refers to the configuration value stored in the period register of the PWM timer selected by the PWM operator (i.e., [MCPWM_TIMERn_PERIOD](#) ($x = 0, 1, 2$)). For configuration methods, please refer to the [section 56.3.2](#). A and B denote the time stamp registers A and B used by the hardware. For register configuration, please refer to the [section 56.3.3.1](#). UTEA/B, UTEZ/P, DTEA/B, DTEZ/P, DTO/1, and UTO/1 represent different timing events of the PWM generator. For event definition, please refer to the [section 56.3.3.1](#). The upward arrow, downward arrow, and T mark signify setting the PWM waveform high, setting it low, toggling it when the timing event occurs. See [section 56.3.3.1](#) for configuration. PWMxA and PWMxB are the output signals of PWM generatorx.

Figure 56.3-13 presents the symmetric PWM waveform generated when the PWM timer is in Count-Up-Down mode. DC 0%–100% modulation can be calculated via the formula below:

$$Duty = (Period - A) \div Period$$

If A matches the PWM timer value and the PWM timer is incrementing, then the PWM output is pulled up. If A matches the PWM timer value while the PWM timer is decrementing, then the PWM output is pulled low.

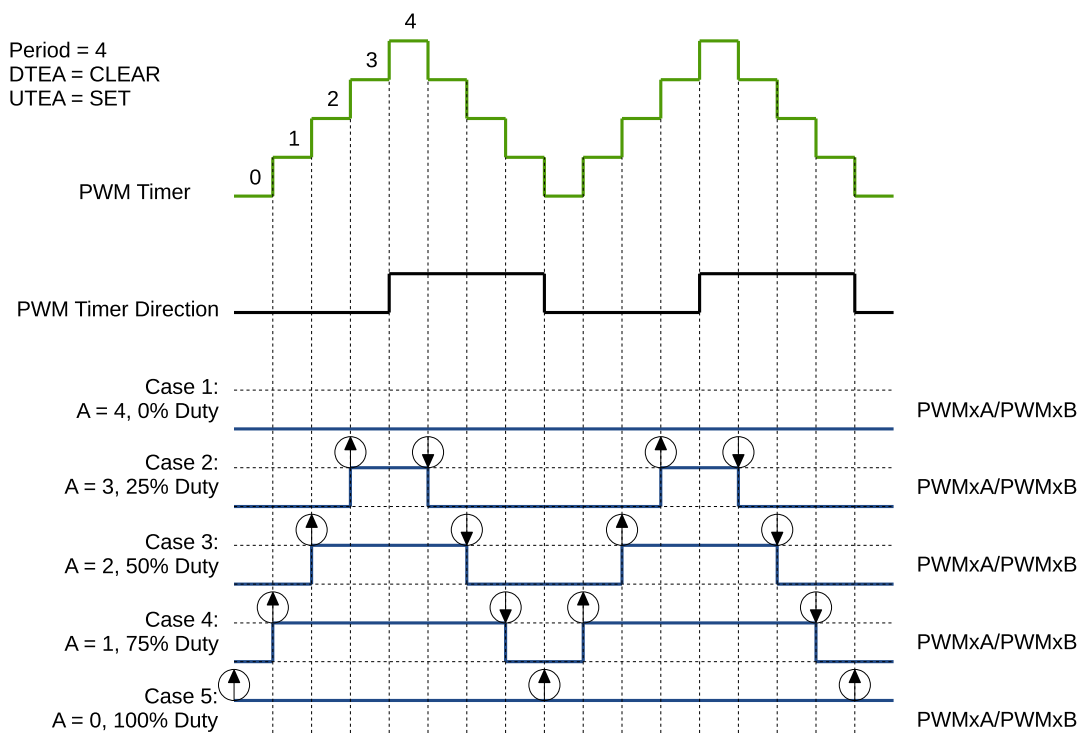


Figure 56.3-13. Symmetrical Waveform in Count-Up-Down Mode

The PWM waveforms in [Figures 56.3-14 to 56.3-17](#) show some common PWM generator configurations.

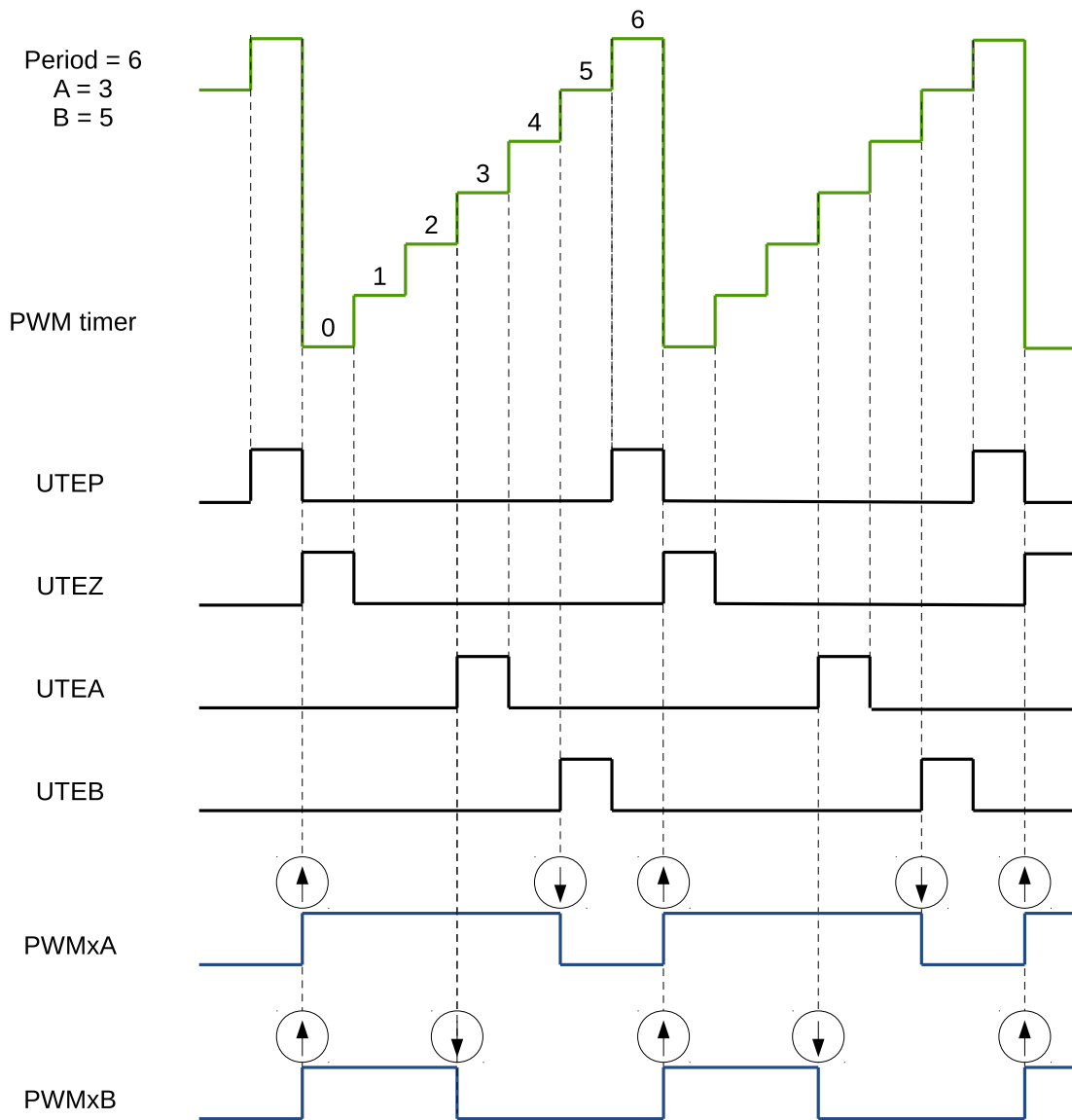


Figure 56.3-14. Count-Up, Single Edge Asymmetric Waveform, with Independent Modulation on PWMxA and PWMxB — Active High

The duty modulation for PWMxA is set by B, active high and proportional to B.

The duty modulation for PWMxB is set by A, active high and proportional to A.

$$Period = (MCPWM_TIMER_PERIOD + 1) \times T_{PT_clk}$$

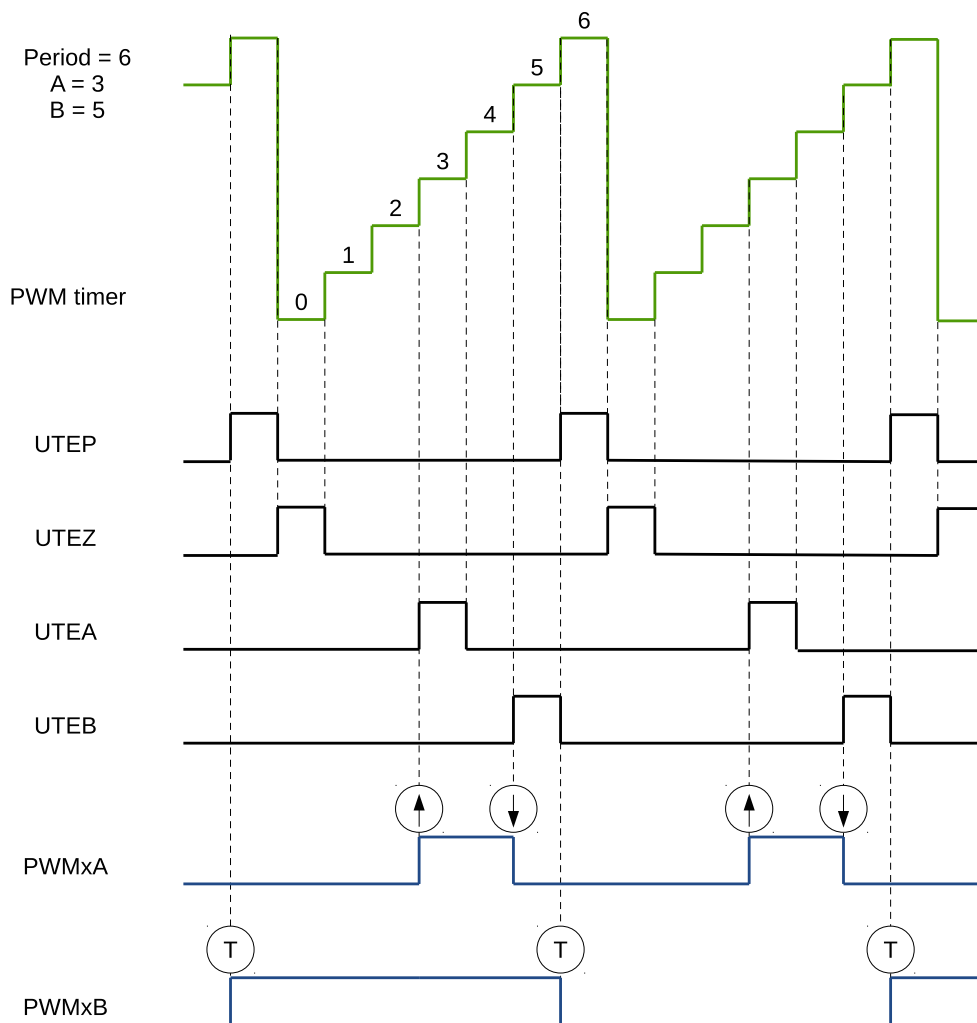


Figure 56.3-15. Count-Up, Pulse Placement Asymmetric Waveform with Independent Modulation on PWMxA

Pulses may be generated anywhere within the PWM cycle (zero to period).

PWMxA's high time duty is proportional to (B - A).

$$Period = (MCPWM_TIMERn_PERIOD + 1) \times T_{PT_clk}$$

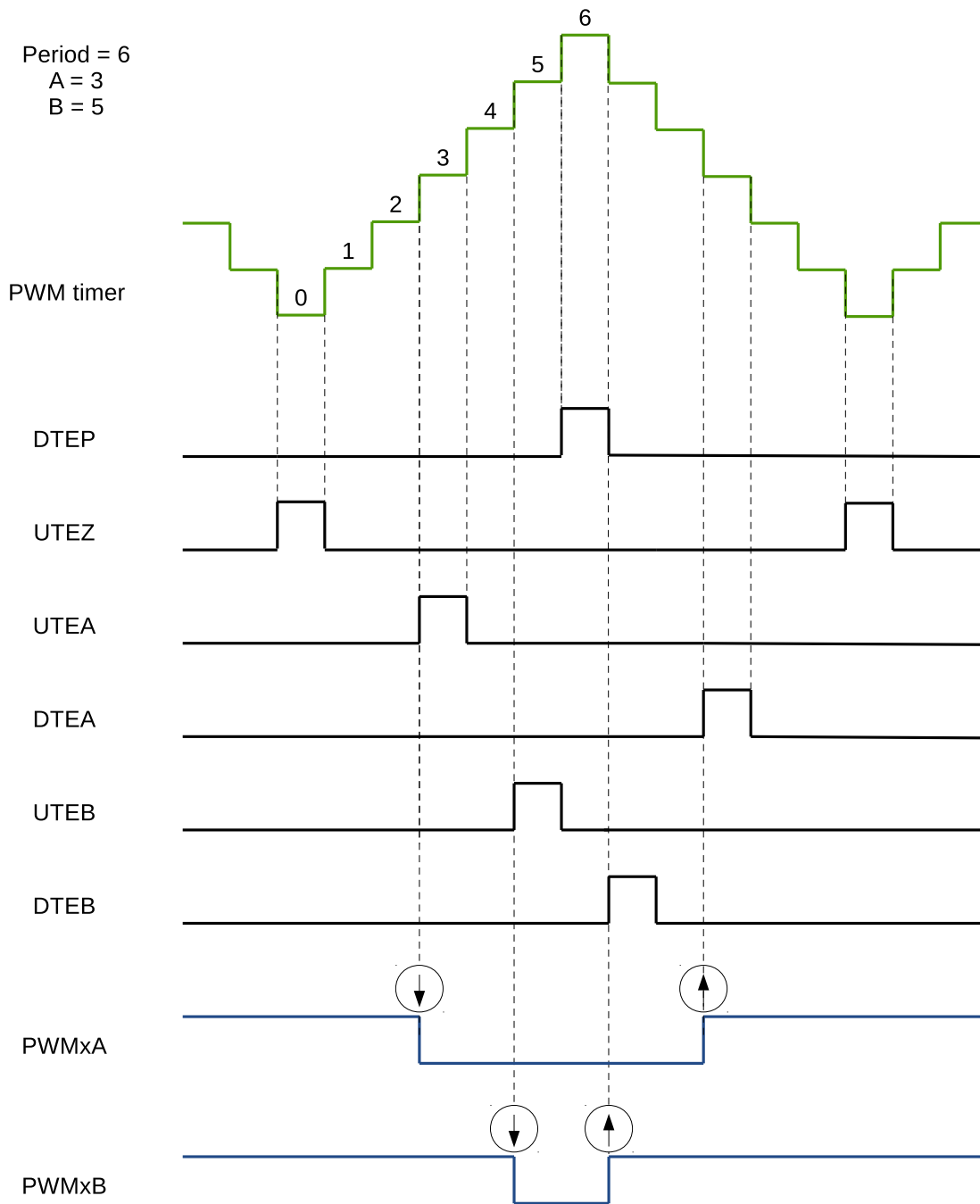


Figure 56.3-16. Count-Up-Down, Dual Edge Symmetric Waveform, with Independent Modulation on PWMxA and PWMxB — Active High

The duty modulation for PWMxA is set by A, active high and proportional to A.
The duty modulation for PWMxB is set by B, active high and proportional to B.
Outputting PWMxA and PWMxB can drive separate switches.

$$Period = (2 \times MCPWM_TIMER_PERIOD) \times T_{PT_clk}$$

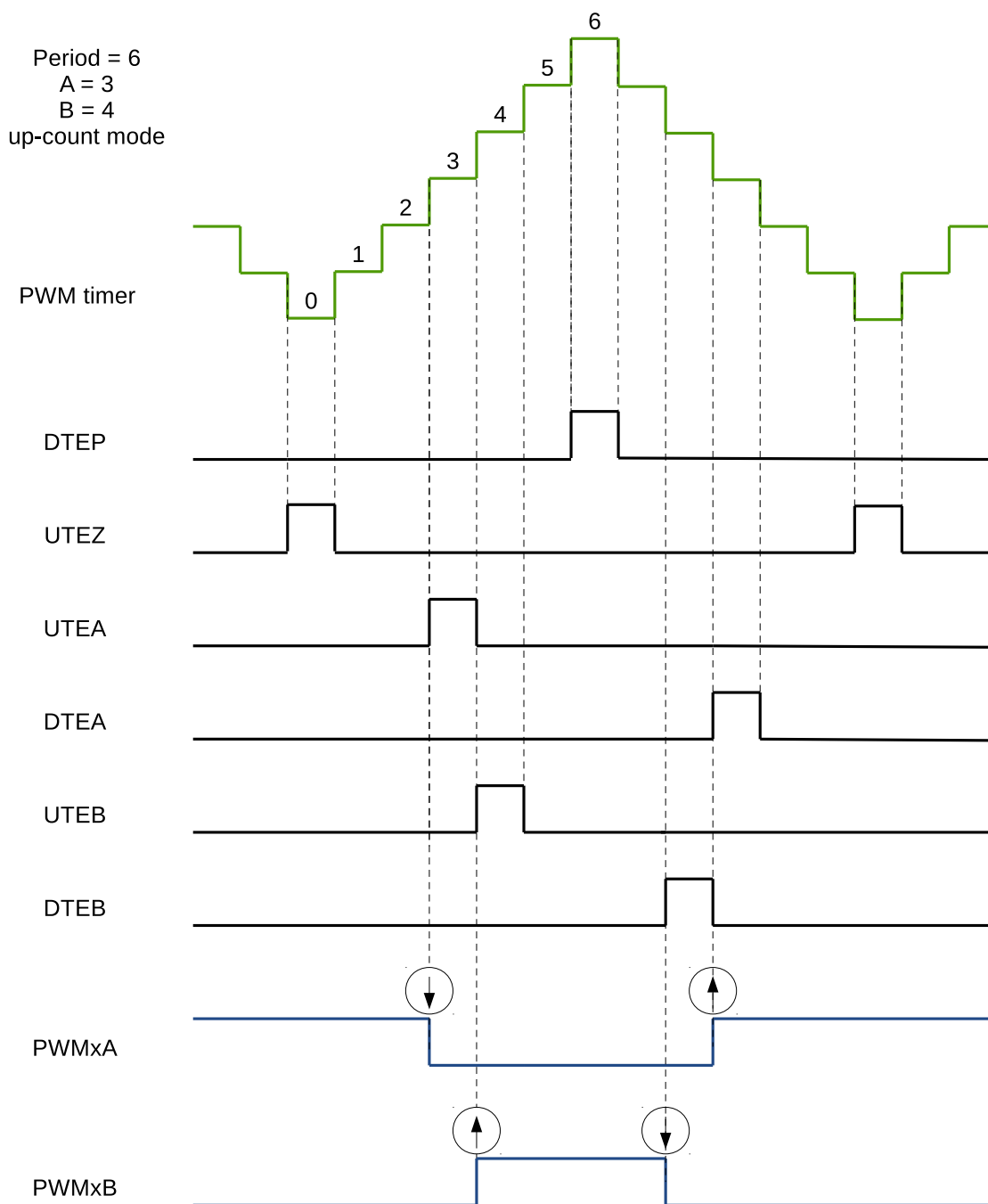


Figure 56.3-17. Count-Up-Down, Dual Edge Symmetric Waveform, with Independent Modulation on PWMxA and PWMxB – Complementary

The duty modulation of PWMxA is set by A, is active high and proportional to A.

The duty modulation of PWMxB is set by B, is active low and proportional to B.

Outputs PWMxA can drive upper/lower (complementary) switches.

Dead time = B – A. Edge placement is configurable by software. Dead time generator module supports configuring edge delay methods when required.

$$Period = (2 \times MCPWM_TIMERn_PERIOD) \times T_{PT_clk}$$

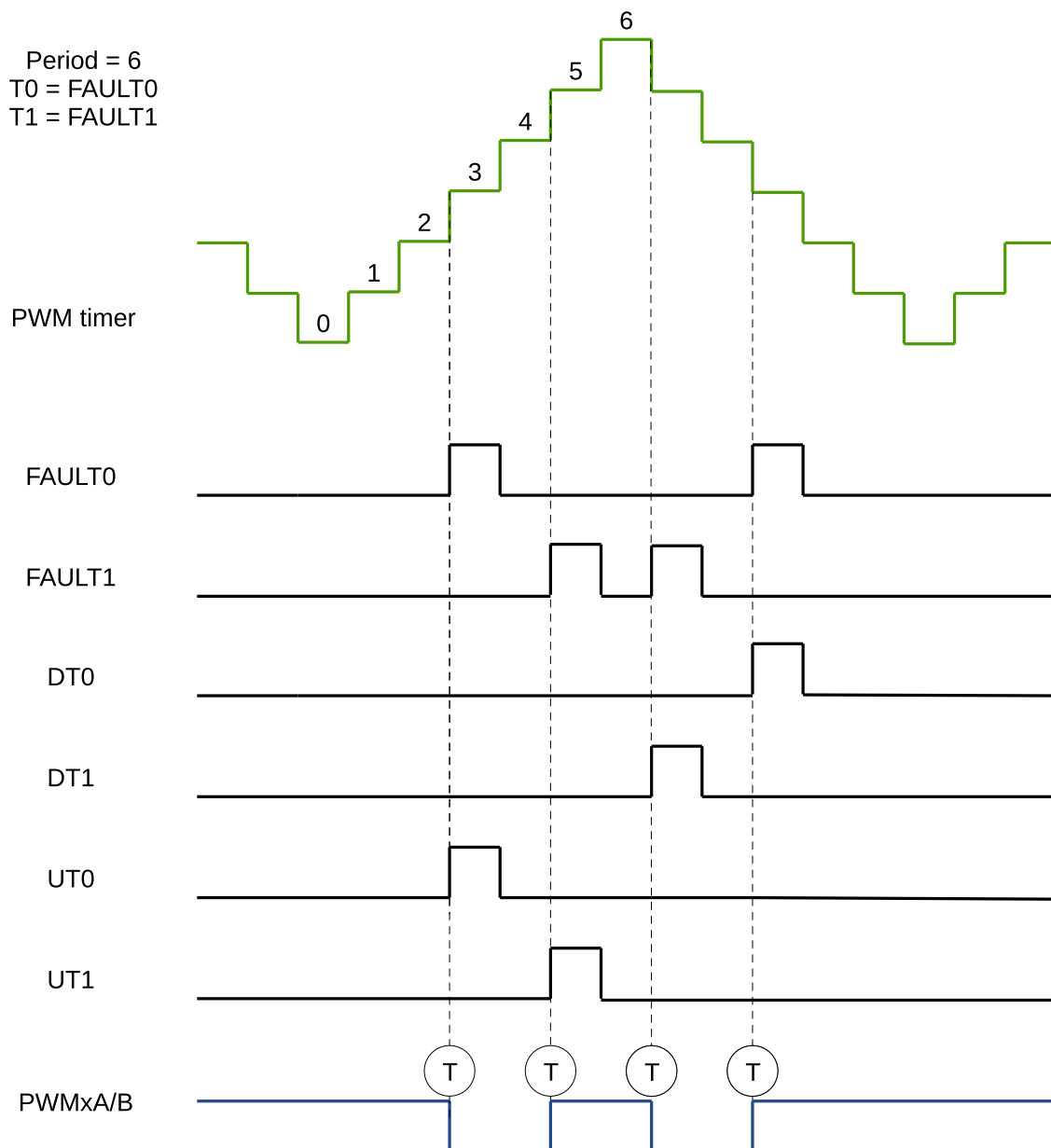


Figure 56.3-18. Count-Up-Down, Fault or Synchronization Events, with Same Modulation on PWMxA and PWMxB

Figure 56.3-18 shows a waveform when UT0/1 and DT0/1 events are generated. In this example, T0 selects FAULT0 and T1 selects FAULT1. The events selected by T0 and T1 can be configured independently, these events can be FAULT0, FAULT1, FAULT2 or synchronous. For detailed configuration, see section 56.3.3.1.

Software-Force Events

There are two types of software-force events inside the PWM generator:

- Non-continuous-immediate (NCI) software-force events: Such types of events are immediately effective on PWM outputs when triggered by software. The forcing is non-continuous, which means the next active timing events will be able to alter the PWM outputs.
- Continuous (CNTU) software-force events: Such types of events are continuous. The forced PWM

outputs will continue until they are released by software. The events' triggers are configurable. They can be configured to be timing events or immediate events.

For NCI software-force events, set register `MCPWM_GENn_A_NCIFORCE` or `MCPWM_GENn_B_NCIFORCE` to 1 and then to 0 to trigger the NCI software-forced events of PWMxA or PWMxB. Configure register `MCPWM_GENn_A_NCIFORCE_MODE` or `MCPWM_GENn_B_NCIFORCE_MODE` to specify the operation on the PWMxA or PWMxB waveform when the NCI software-force event occurs. Configuring the registers to 0 or 3 means no operation, 1 means low level, and 2 means high level.

For CNTU software-force events, set register `MCPWM_GENn_CNTUFORCE_UPMETHOD` to specify the trigger method of CNTU software-force events. Setting all bits to 0 means trigger CNTU software-force events immediately, setting bit 0 to 1 means the TEZ event triggers CNTU software-force events, setting bit 1 to 1 means the TEP event triggers CNTU software-force events, setting bit 2 to 1 means the TEA event triggers CNTU software-force events, setting bit 3 to 1 means the TEB event triggers CNTU software-force events, setting bit 4 to 1 means CNTU software-force events triggered when the PWM timer selected by the PWM generator is synchronized, and setting bit 5 to 1 means the CNTU software-forced events will not be triggered. Configure register `MCPWM_GENn_A_CNTUFORCE_MODE` or `MCPWM_GENn_B_CNTUFORCE_MODE` to specify the operation on the PWMxA or PWMxB waveform when the CNTU software-force events occur. Configuring the registers to 0 or 3 means no operation, 1 means low level, and 2 means high level.

Figure 56.3-19 shows a waveform of NCI software-force events. NCI events are used to force PWMxA output low. Forcing on PWMxB is disabled in this case.

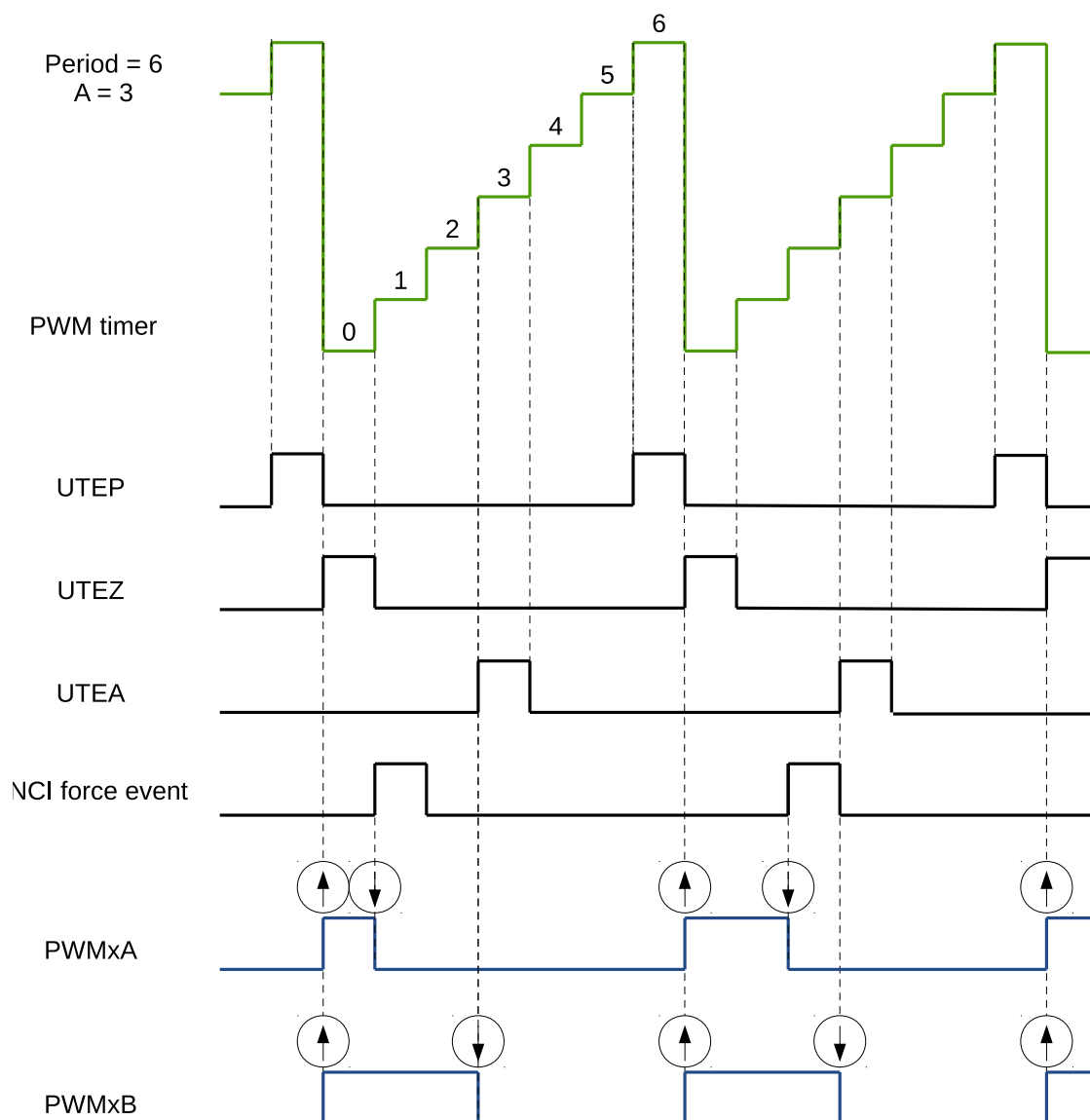


Figure 56.3-19. Example of an NCI Software-Force Event on PWMxA

Figure 56.3-20 shows a waveform of CNTU software-force events. TEZ events are selected as triggers for CNTU software-force events. CNTU is used to force the PWMxB output low. Forcing on PWMxA is disabled.

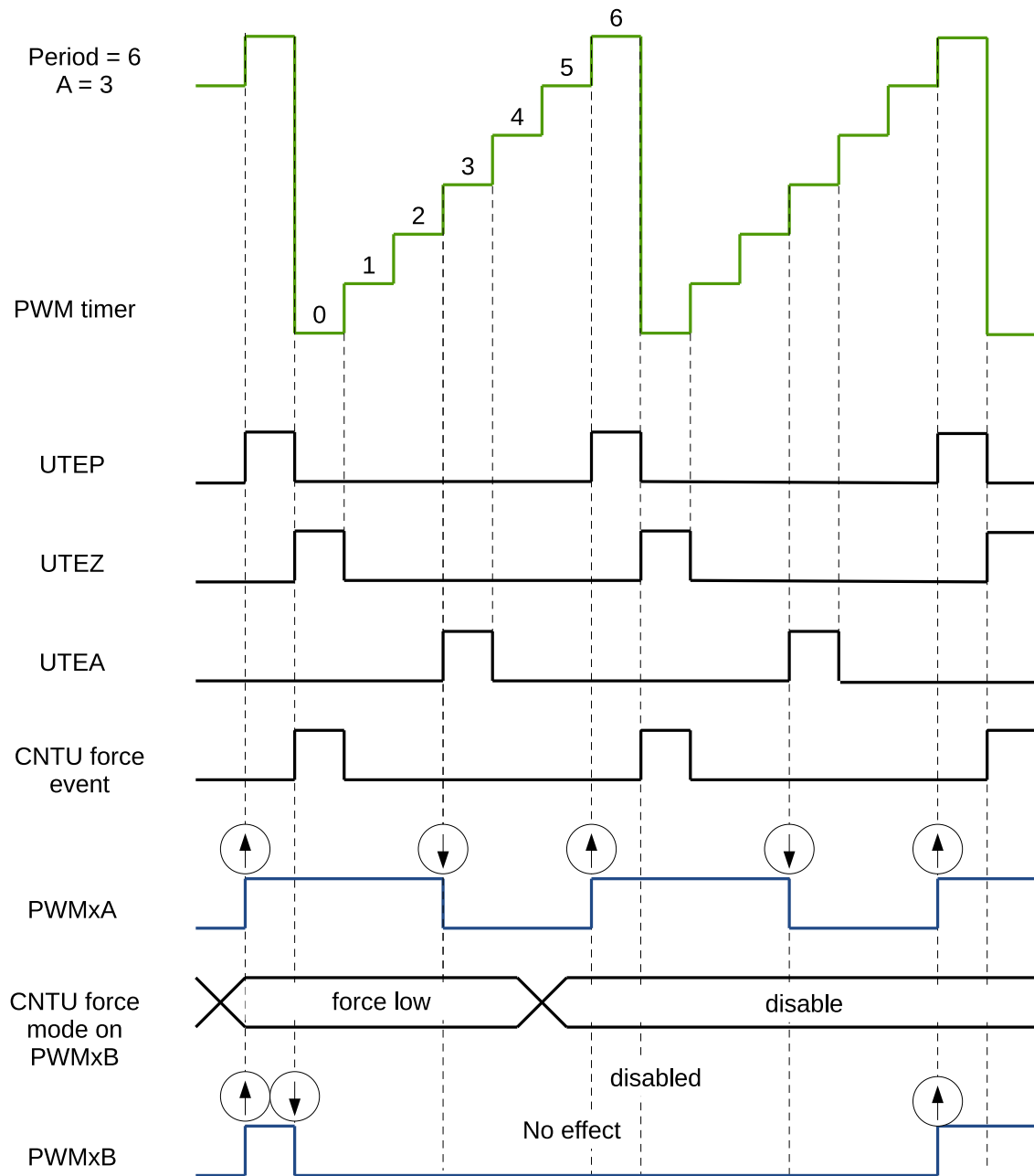


Figure 56.3-20. Example of a CNTU Software-Force Event on PWMxB

56.3.3.2 Dead Time Generator Module

Purpose of the Dead Time Generator Module

Section 56.3.3.1 introduced several options to generate signals on PWMxA and PWMxB outputs, with a specific placement of signal edges. The required dead time is obtained by altering the edge placement between signals and by setting the signal's duty cycle. Another option to control the dead time is to use a specialized module – Dead Time Generator.

The key functions of the Dead Time Generator module are as follows:

- Generating output signal pairs (PWMxA and PWMxB) with a dead time from a common source, which can be either the input signal PWMxA or PWMxB.
- Creating a dead time by adding delay to signal edges:
 - Rising edge delay (RED)
 - Falling edge delay (FED)
- Can generate PWM signal in various format. The typical dead time configurations are:
 - Active high complementary (AHC)
 - Active low complementary (ALC)
 - Active high (AH)
 - Active low (AL)
- This module may also be bypassed, if the dead time is configured directly in the generator module.

Shadow Register of Dead Time Generator

Delay registers RED and FED are shadowed with registers `MCPWM_DT $n$ _RED_CFG_REG` and `MCPWM_DT $n$ _FED_CFG_REG`. When `MCPWM_GLOBAL_UP_EN` is set to 1, the values saved in the shadow registers can be written to the active register at the specified time. The update method register for `MCPWM_DT $n$ _RED_CFG_REG` is `MCPWM_DB $n$ _RED_UPMETHOD`. The update method register for `MCPWM_DT $n$ _FED_CFG_REG` is `MCPWM_DB $n$ _FED_UPMETHOD`. The Software can also trigger a globally forced update bit `MCPWM_GLOBAL_FORCE_UP` which will prompt all registers in the module to be updated according to shadow registers. For the description of shadow registers, please see section 56.3.2.3.

Highlights for Operation of the Dead Time Generator

Options for setting up the dead time module are shown in Figure 56.3-21.

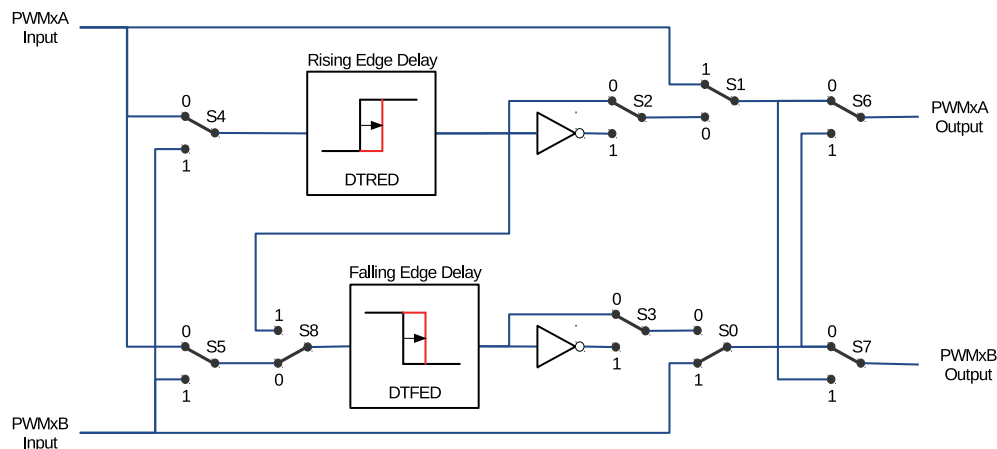


Figure 56.3-21. Options for Setting up the Dead Time Generator Module

S0-S8 in the figure above are switches controlled by fields in register `MCPWM_DTn_CFG_REG` shown in Table 56.3-5.

Table 56.3-5. Dead Time Generator Switches Control Fields

Switch	Field
S0	<code>MCPWM_DBn_B_OUTBYPASS</code>
S1	<code>MCPWM_DBn_A_OUTBYPASS</code>
S2	<code>MCPWM_DBn_RED_OUTINVERT</code>
S3	<code>MCPWM_DBn_FED_OUTINVERT</code>
S4	<code>MCPWM_DBn_RED_INSEL</code>
S5	<code>MCPWM_DBn_FED_INSEL</code>
S6	<code>MCPWM_DBn_A_OUTSWAP</code>
S7	<code>MCPWM_DBn_B_OUTSWAP</code>
S8	<code>MCPWM_DBn_DEB_MODE</code>

All switch combinations are supported, but not all of them represent the typical modes of use. Table 56.3-6 documents some typical dead time configurations. In these configurations, the position of S4 and S5 sets PWMxA as the common source of both falling edge delay (FED) and rising edge delay (RED). The modes presented in table 56.3-6 may be categorized as follows:

Table 56.3-6. Typical Dead Time Generator Operating Modes

Mode	Mode Description	S0	S1	S2	S3
1	PWMxA and PWMxB Pass Through/No Delay	1	1	X	X
2	Active High Complementary (AHC), see Figure 56.3-22	0	0	0	1
3	Active Low Complementary (ALC), see Figure 56.3-23	0	0	1	0
4	Active High (AH), see Figure 56.3-24	0	0	0	0

Mode	Mode Description	S0	S1	S2	S3
5	Active Low (AL), see Figure 56.3-25	0	0	1	1
6	PWMxA Output = PWMxA In (No Delay) PWMxB Output = PWMxA Input with Falling Edge Delay	0	1	0 or 1	0 or 1
7	PWMxA Output = PWMxA Input with Rising Edge Delay PWMxB Output = PWMxB Input with No Delay	1	0	0 or 1	0 or 1

Note:

For all the modes above, the position of the binary switches S4 to S8 is set to 0.

- **Mode 1: Bypass delays on both FED and RED**

In this mode, the dead time module is disabled. Signals of PWMxA and PWMxB pass through without any modifications.

- **Mode 2-5: Classical Dead Time Polarity Settings**

These four modes represent typical configurations of polarity and should cover the active-high/low modes in available industry power switch gate drivers. The typical waveforms are shown in Figures 56.3-22 to 56.3-25.

- **Modes 6 and 7: Bypass delay on the falling edge (FED) or rising edge (RED)**

In these two modes, either RED or FED is bypassed. As a result, the corresponding delay is not applied.

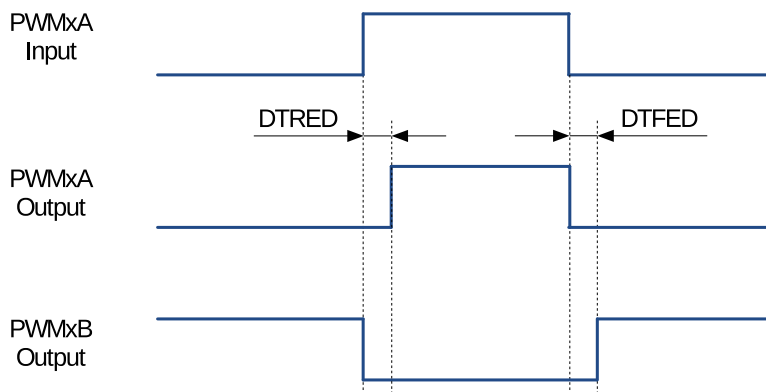


Figure 56.3-22. Active High Complementary (AHC) Dead Time Waveforms

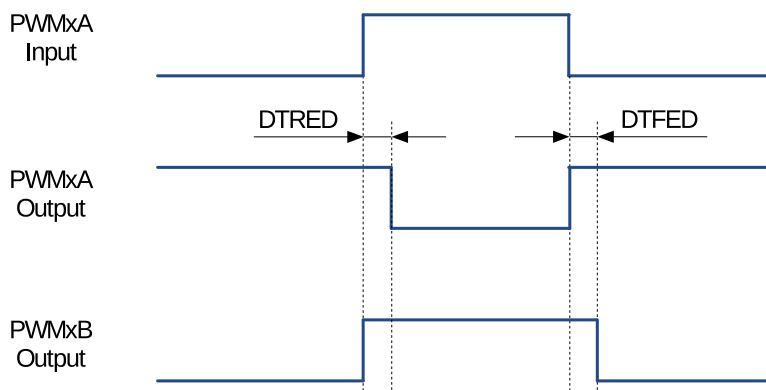


Figure 56.3-23. Active Low Complementary (ALC) Dead Time Waveforms

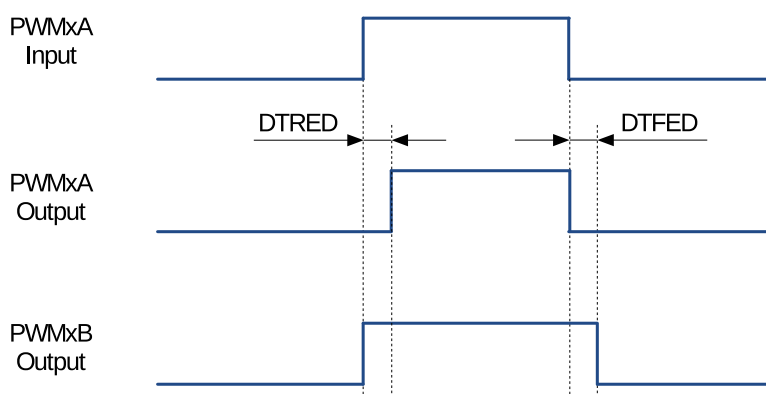


Figure 56.3-24. Active High (AH) Dead Time Waveforms

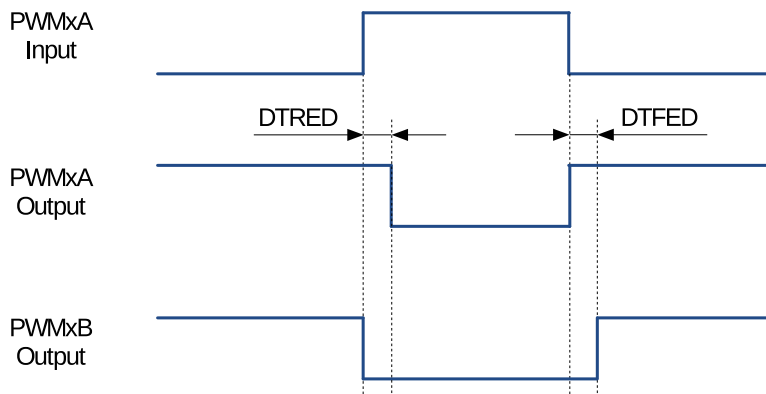


Figure 56.3-25. Active Low (AL) Dead Time Waveforms

RED and FED delays may be set up independently. The delay value is programmed using the 16-bit field `MCPWM_DB $n$ _RED` and `MCPWM_DB $n$ _FED`. The register value represents the number of clock (DT_CLK) periods by which a signal edge is delayed. DT_CLK can be selected from PWM_clk or PT_clk through register `MCPWM_DB $n$ _CLK_SEL`.

To calculate the delay on the falling edge (FED) and rising edge (RED), use the following formulas:

$$FED = MCPWM_DTn_FED \times T_{DT_clk}$$

$$RED = MCPWM_DTn_RED \times T_{DT_clk}$$

56.3.3.3 PWM Carrier Module

The coupling of PWM output to a motor driver may need isolation with a transformer. Transformers deliver only AC signals, while the duty cycle of a PWM signal may range anywhere from 0% to 100%. The PWM carrier module passes such a PWM signal through a transformer by using a high frequency carrier to modulate the signal.

Function Overview

The following key characteristics of this module are configurable:

- Carrier frequency
- Pulse width of the first pulse
- Duty cycle of the second and the subsequent pulses
- Enabling/disabling the carrier function

Operational Highlights

The PWM carrier clock (PC_clk) is derived from PWM_CLK. The frequency and duty cycle are configured by the `MCPWM_CHOPPERn_PRESCALE` and `MCPWM_CHOPPERn_DUTY` bits in the `MCPWM_CARRIERn_CFG_REG` register. The purpose of one-shot pulses is to provide the high-energy impulse to reliably turn on the power switch. Subsequent pulses sustain the power-on status. The width of a one-shot pulse is configurable with the `MCPWM_CHOPPERn_OSHTWTH` field. Enabling/disabling of the carrier module is done with the `MCPWM_CHOPPERn_EN` bit.

Waveform Examples

Figure 56.3-26 shows an example of waveforms, where a carrier is superimposed on original PWM pulses. This figure does not show the first one-shot pulse and the duty-cycle control. Related details are covered in the following two sections.

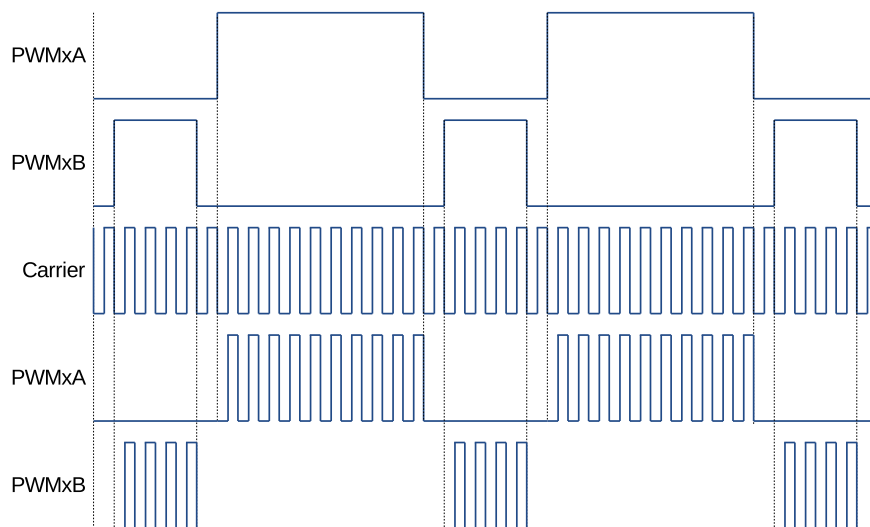


Figure 56.3-26. Example of Waveforms Showing PWM Carrier Action

One-Shot Pulse

The width of the first pulse can be configured to 16 different values, which can be calculated by the following equation:

$$T_{1stpulse} = T_{PWM_clk} \times 8 \times (MCPWM_CARRIERn_PRESCALE + 1) \times (MCPWM_CARRIERn_OSHTWTH + 1)$$

Where:

- T_{PWM_clk} is the period of the PWM clock (PWM_clk).
- $(MCPWM_CARRIERn_OSHTWTH + 1)$ is the width of the first pulse (whose value ranges from 1 to 16).
- $(MCPWM_CARRIERn_PRESCALE + 1)$ is the PWM carrier clock's (PC_clk) prescaler value.

The first one-shot pulse and subsequent sustaining pulses are shown in Figure 56.3-27.

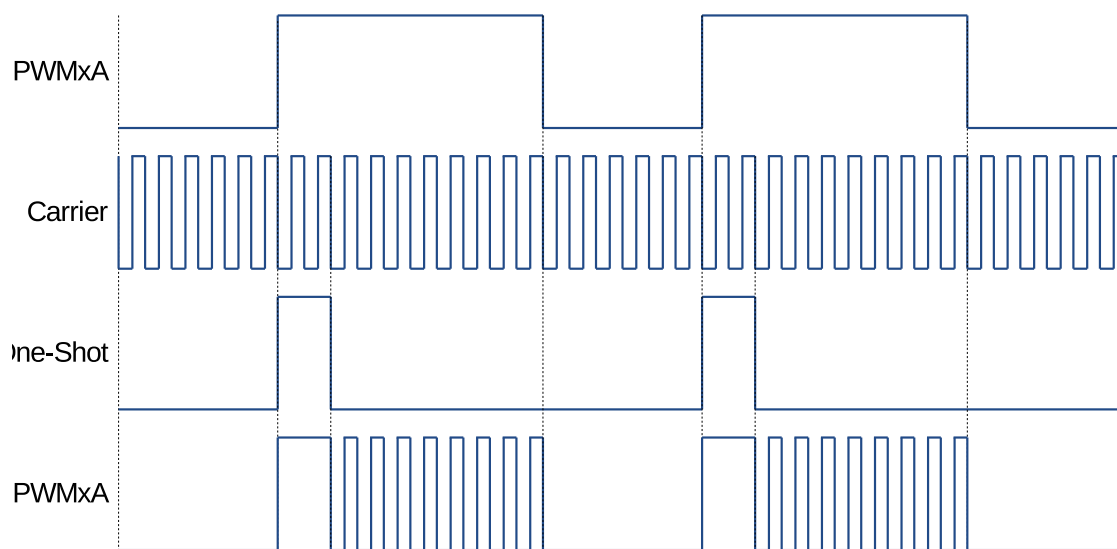


Figure 56.3-27. Example of the First Pulse and the Subsequent Sustaining Pulses of the PWM Carrier Submodule

Duty Cycle Control

After issuing the first one-shot pulse, the remaining PWM signal is modulated according to the carrier frequency. Users can configure the duty cycle of this signal. Tuning of duty may be required, so that the signal passes through the isolating transformer and can still operate (turn on/off) the motor drive, changing rotation speed and direction.

The duty cycle may be set to one of seven values, using `MCPWM_CHOPPERn_DUTY`, or bits [7:5] of register `MCPWM_CARRIERn_CFG_REG`.

Below is the formula for calculating the duty cycle:

$$Duty = MCPWM_CARRIERn_DUTY \div 8$$

All seven settings of the duty cycle are shown in Figure 56.3-28.

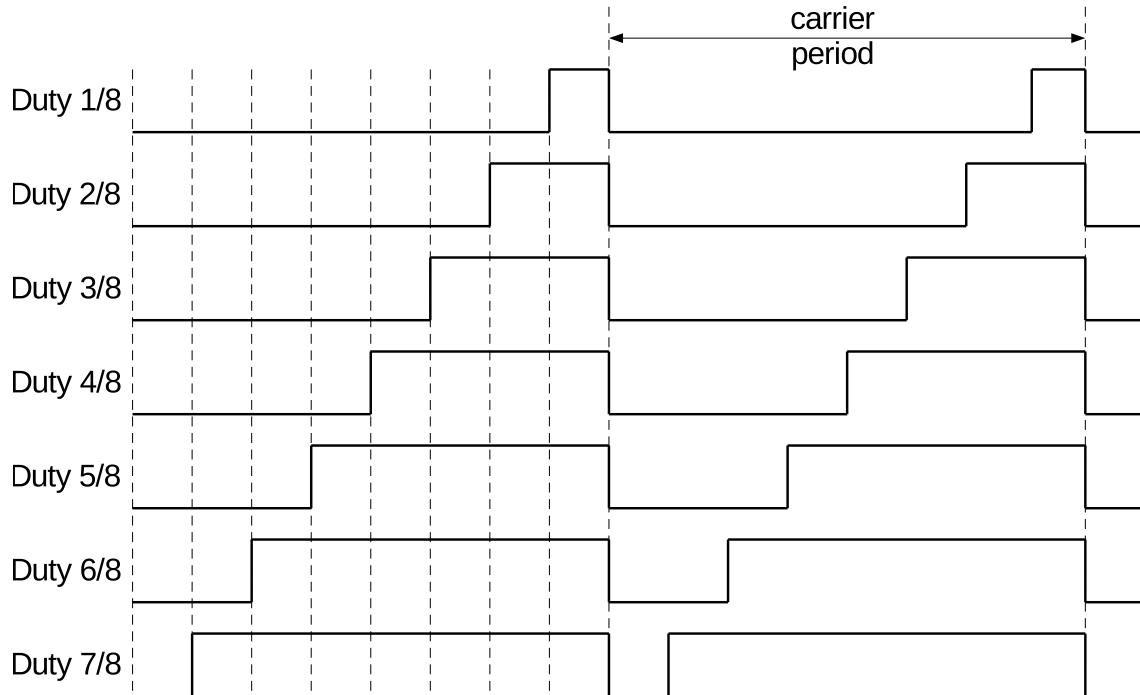


Figure 56.3-28. Possible Duty Cycle Settings for Sustaining Pulses in the PWM Carrier Submodule

56.3.3.4 Fault Detection Module

Each MCPWM peripheral is connected to three fault signals (FAULT0, FAULT1, and FAULT2) which are sourced from the GPIO matrix. These signals are intended to indicate external fault conditions, and may be preprocessed by the Fault Detection module to generate fault events. Fault events can then execute the user code to control MCPWM outputs in response to specific faults.

Function of Fault Detection Module

The key actions performed by the fault detection module are:

- Forcing outputs PWMxA and PWMxB, upon detected fault, to one of the following states:
 - High
 - Low
 - Toggle
 - No action taken
- Execution of one-shot trip (OST) upon detection of over-current conditions/short circuits.
- Cycle-by-cycle trip (CBC) to provide current-limiting operation.
- Allocation of either one-shot or cycle-by-cycle operation for each fault signal.
- Generation of interrupts for each fault input.
- Support for software-force tripping.
- Enabling or disabling of module function as required.

Operation and Configuration Tips

This section provides the operational tips and set-up options for the Fault Detection module.

Fault signals coming from pins are sampled and synced in the GPIO matrix. In order to guarantee the successful sampling of fault pulses, each pulse duration must be at least two APB clock cycles. The Fault Detection module will then sample fault signals by using PWM_clk. So, the duration of fault pulses coming from GPIO matrix must be at least one PWM_clk cycle. Differently put, regardless of the period relation between APB clock and PWM_clk, the width of fault signal pulses on pins must be at least equal to the sum of two APB clock cycles and one PWM_clk cycle.

Each level of fault signals, FAULT0 to FAULT2, can be used by the Fault Detection module to generate fault events (fault_event0 to fault_event2). Every fault event can be configured individually to provide CBC action, OST action, or none.

- **Cycle-by-Cycle (CBC) action:**

When CBC action is triggered, the state of PWMxA and PWMxB will be changed immediately according to the configuration of fields [MCPWM_TZn_A_CBC_U/D](#) and [MCPWM_TZn_B_CBC_U/D](#). Different actions can be indicated when the PWM timer is incrementing or decrementing. Different CBC action interrupts can be triggered for different fault events. Status field [MCPWM_TZn_CBC_ON](#) indicates whether a CBC action is on or off. When the fault event is no longer present, CBC actions on PWMxA/B will be cleared at a specified point, which is either a D/UTEP or D/UTEZ event. Field [MCPWM_TZn_CBCPULSE](#) determines at which event PWMxA and PWMxB will be able to resume normal actions. Therefore, in this mode, the CBC action is cleared or refreshed upon every PWM cycle.

- **One-Shot (OST) action:**

When OST action is triggered, the state of PWMxA and PWMxB will be changed immediately, depending on the setting of fields [MCPWM_TZn_A_OST_U/D](#) and [MCPWM_TZn_B_OST_U/D](#). Different actions can be configured when PWM timer is incrementing or decrementing. Different OST action interrupts can be triggered from different fault events. Status field [MCPWM_TZn_OST_ON](#) indicates whether an OST action is on or off. The OST actions on PWMxA/B are not automatically cleared when the fault event is no longer present. One-shot actions must be cleared manually by setting the [MCPWM_TZn_CLR_OST](#) bit.

56.3.4 Capture Module

56.3.4.1 Introduction

The capture module contains three complete capture channels. Channel inputs CAPO, CAP1, and CAP2 are sourced from the GPIO matrix. Thanks to the flexibility of the GPIO matrix, CAPO, CAP1, and CAP2 can be configured from any pin input. Multiple capture channels can be sourced from the same pin input, while prescaling for each channel can be set differently. Also, capture channels are sourced from different pins. This provides several options for handling capture signals by hardware in the background, instead of having them processed directly by the CPU. A capture module has the following independent key resources:

- One 32-bit timer (counter) which can be synchronized with the PWM timer, another module, or software.
- Three capture channels, each equipped with a 32-bit time-stamp and a capture prescaler.
- Independent edge polarity (rising/falling edge) selection for any capture channel.
- Input capture signal prescaling (from 1 to 256).

- Interrupt capabilities on any of the three capture events.

56.3.4.2 Capture Timer

The capture timer is a 32-bit counter incrementing continuously. It is enabled by setting `MCPWM_CAP_TIMER_EN` to 1. Its operating clock source is MCPWM core clock. When `MCPWM_CAP_SYNCI_EN` is configured, the counter will be loaded with phase stored in register `MCPWM_CAP_TIMER_PHASE_REG` at the time of a sync event. Sync events can select from PWM timers sync-out, or PWM module sync-in by configuring `MCPWM_CAP_SYNCI_SEL`. Sync events can also be generated by setting `MCPWM_CAP_SYNC_SW`. The capture timer provides timing references for all three capture channels.

56.3.4.3 Capture Channel

The capture signal coming to a capture channel will be inverted first, if needed, and then prescaled. Each capture channel has a prescaler register of `MCPWM_CAP $n$ _PRESCALE`. Finally, specified edges of preprocessed capture signal will trigger capture events. Setting `MCPWM_CAP $n$ _EN` to enable a capture channel. The capture event occurs at the time selected by the `MCPWM_CAP $n$ _MODE`. When a capture event occurs, the capture timer's value is stored in time-stamp register `MCPWM_CAP_CH $n$ _REG`. Different interrupts can be generated for different capture channels at capture events. The edge that triggers a capture event is recorded in register `MCPWM_CAP $n$ _EDGE`. The capture event can be also forced by software setting `MCPWM_CAP $n$ _SW`.

56.3.5 ETM Module

56.3.5.1 Overview

The MCPWM peripheral on ESP32-P4 supports the Event Task Matrix (ETM) function, which allows MCPWM's ETM tasks to be triggered by any peripherals' ETM events, or MCPWM's ETM events to trigger any peripherals' ETM tasks. The capture module, the fault detection module, three timers, and three operators can generate events and respond to tasks independently. This section introduces the ETM tasks and events related to MCPWM. For more information, please refer to Chapter 13 [Event Task Matrix \(ETM\)](#).

56.3.5.2 MCPWM-Related ETM Events

When setting enable field to 1, after the generation condition is met, the corresponding event would be generated. For details, please refer to Table 56.3-7 below:

Table 56.3-7. MCPWM-Related ETM Events

Enable Field	Generation Condition	Event Generated
<code>MCPWM_EVT_CAPn_EN</code>	CAP n capture event occurs	<code>MCPWMm_EVT_CAPn</code>
<code>MCPWM_EVT_TZn_OST_EN</code>	PWM operator n performs a One-Shot trip (OST) operation	<code>MCPWMm_EVT_TZn_OST</code>
<code>MCPWM_EVT_TZn_CBC_EN</code>	PWM operator n performs a cycle-by-cycle trip (CBC) operation	<code>MCPWMm_EVT_TZn_CBC</code>
<code>MCPWM_EVT_Fn_CLR_EN</code>	Fault event fault_event n is cleared	<code>MCPWMm_EVT_Fn_CLR</code>

¹ See Section 56.3.3.1 for a detailed description of timer stamp A and B.

Enable Field	Generation Condition	Event Generated
MCPWM_EVT_Fn_EN	Fault event fault_event n is generated	MCPWM m _EVT_Fn
MCPWM_EVT_OPn_TEB_EN	The count value of the timer that PWM operator n selects is equal to the value of timer stamp B ¹	MCPWM m _EVT_OPn_TEB
MCPWM_EVT_OPn_TEA_EN	The count value of the timer that PWM operator n selects is equal to the value of timer stamp A ¹	MCPWM m _EVT_OPn_TEA
MCPWM_EVT_OPn_TEE1_EN	The count value of the timer that PWM operator n selects is equal to the value of register MCPWM_OPn_TSTMP_E1_REG	MCPWM m _EVT_OPn_TEE1
MCPWM_EVT_OPn_TEE2_EN	The count value of the timer that PWM operator n selects is equal to the value of register MCPWM_OPn_TSTMP_E2_REG	MCPWM m _EVT_OPn_TEE2
MCPWM_EVT_TIMERn_TEP_EN	The count value of timer n is equal to the period value MCPWM_TIMERn_PERIOD	MCPWM m _EVT_TIMERn_TEP
MCPWM_EVT_TIMERn_TEZ_EN	The count value of timer n is equal to 0	MCPWM m _EVT_TIMERn_TEZ
MCPWM_EVT_TIMERn_STOP_EN	Timer n 's count stops	MCPWM m _EVT_TIMERn_STOP

¹ See Section 56.3.3.1 for a detailed description of timer stamp A and B.

56.3.5.3 MCPWM-Related ETM Tasks

When setting the enable field to 1, after inputting valid tasks, the corresponding response operation would be generated. For details, please refer to Table 56.3-8 below:

Table 56.3-8. ETM Related Tasks

Enable Field	Valid Task Input	Response Operation
MCPWM_TASK_CAPn_EN	MCPWM m _TASK_CAPn	CAP n channel performs a capture operation
MCPWM_TASK_CLRn_OST_EN	MCPWM m _TASK_CLRn_OST	PWM operator n clears the One-Shot Trip operation
MCPWM_TASK_TZn_OST_EN	MCPWM m _TASK_TZn_OST	PWM operator n performs a One-Shot Trip (OST) operation
MCPWM_TASK_TIMERn_PERIOD_UP_EN	MCPWM m _TASK_TIMERn_PERIOD_UP	The period of timer n is updated to the value configured in the period register MCPWM_TIMERn_PERIOD
MCPWM_TASK_TIMERn_SYNC_EN	MCPWM m _TASK_TIMERn_SYNC	Timer n performs a sync operation
MCPWM_TASK_GEN_STOP_EN	MCPWM m _TASK_GEN_STOP	All the timers stop counting and the PWM signals output by all PWM operators remain unchanged

Enable Field	Valid Task Input	Response Operation
MCPWM_TASK_CMPR n _B_UP_EN	MCPWM m _TASK_CMPR n _B_UP	Timer stamp B of the PWM operator n is updated to the value of the shadow register MCPWM_GEN n _B
MCPWM_TASK_CMPR n _A_UP_EN	MCPWM m _TASK_CMPR n _A_UP	Timer stamp A of the PWM operator n is updated to the value of the shadow register MCPWM_GEN n _A

56.4 Interrupts

ESP32-P4's MCPWM0 can generate the PWM0_INTR interrupt signal and MCPWM1 can generate the PWM1_INTR interrupt signal. The PWM n _INTR will be sent to the [Interrupt Matrix](#).

Interrupt signal PWM n _INTR can be generated by the following internal interrupt sources:

- MCPWM_TIMER x _STOP_INT: Triggered when timer x stops.
- MCPWM_TIMER x _TEZ_INT: Triggered by the TEZ event of PWM timer x .
- MCPWM_TIMER x _TEP_INT: Triggered by the TEP event of PWM timer x .
- MCPWM_FAULT x _INT: Triggered when fault_event x starts.
- MCPWM_FAULT x _CLR_INT: Triggered after fault_event x ends.
- MCPWM_CMPR x _TEA_INT: Triggered by the TEA event of PWM operator x .
- MCPWM_CMPR x _TEB_INT: Triggered by the TEB event of PWM operator x .
- MCPWM_TZ x _CBC_INT: Triggered by the CBC action of PWM x .
- MCPWM_TZ x _OST_INT: Triggered by the OST action of PWM x .
- MCPWM_CAP x _INT: Triggered by the capture event on channel x .

The above interrupt sources can only be triggered when the interrupt enable register `interrupt_source_name_ENA` is set to 1. Write 1 to `interrupt_source_name_CLR` will clear the interrupt.

Note:

For definitions of *interrupt*, *interrupt signal*, *interrupt source*, and their correlations, please refer to Chapter [12 Interrupt Matrix](#) > Section [12.2 Interrupt Terminology in ESP32-P4](#).

Each interrupt source can be configured by a common set of registers that are described in Section [Interrupt Configuration Registers](#). The specific registers can be found in Section [56.5 Register Summary](#).

56.5 Register Summary

The addresses in this section are relative to Motor Control PWM base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Prescaler Configuration			
MCPWM_CLK_CFG_REG	PWM clock prescaler register	0x0000	R/W
MCPWM Timer 0 Configuration and Status			
MCPWM_TIMER0_CFG0_REG	PWM timer0 period and update method configuration register	0x0004	R/W
MCPWM_TIMER0_CFG1_REG	PWM timer0 working mode and start/stop control configuration register	0x0008	varies
MCPWM_TIMER0_SYNC_REG	PWM timer0 sync function configuration register	0x000C	R/W
MCPWM_TIMER0_STATUS_REG	PWM timer0 status register	0x0010	RO
MCPWM Timer 1 Configuration and Status			
MCPWM_TIMER1_CFG0_REG	PWM timer1 period and update method configuration register	0x0014	R/W
MCPWM_TIMER1_CFG1_REG	PWM timer1 working mode and start/stop control configuration register	0x0018	varies
MCPWM_TIMER1_SYNC_REG	PWM timer1 sync function configuration register	0x001C	R/W
MCPWM_TIMER1_STATUS_REG	PWM timer1 status register	0x0020	RO
MCPWM Timer 2 Configuration and status			
MCPWM_TIMER2_CFG0_REG	PWM timer2 period and update method configuration register	0x0024	R/W
MCPWM_TIMER2_CFG1_REG	PWM timer2 working mode and start/stop control configuration register	0x0028	varies
MCPWM_TIMER2_SYNC_REG	PWM timer2 sync function configuration register	0x002C	R/W
MCPWM_TIMER2_STATUS_REG	PWM timer2 status register	0x0030	RO
Common Configuration for MCPWM Timers			
MCPWM_TIMER_SYNCI_CFG_REG	Synchronization input selection for three PWM timers	0x0034	R/W
MCPWM_OPERATOR_TIMERSEL_REG	Select specific timer for PWM operators	0x0038	R/W
MCPWM Operator 0 Configuration and Status			
MCPWM_GEN0_STMP_CFG_REG	Transfer status and update method for time stamp registers A and B	0x003C	varies
MCPWM_GEN0_TSTMP_A_REG	Shadow register for register A	0x0040	R/W
MCPWM_GEN0_TSTMP_B_REG	Shadow register for register B	0x0044	R/W
MCPWM_GEN0_CFG0_REG	Fault event T0 and T1 handling	0x0048	R/W
MCPWM_GEN0_FORCE_REG	Permissives to force PWMOA and PWMOB outputs by software	0x004C	R/W

Name	Description	Address	Access
MCPWM_GEN0_A_REG	Actions triggered by events on PWM0A	0x0050	R/W
MCPWM_GEN0_B_REG	Actions triggered by events on PWM0B	0x0054	R/W
MCPWM_DTO_CFG_REG	Dead time type selection and configuration	0x0058	R/W
MCPWM_DTO_FED_CFG_REG	Shadow register for falling edge delay (FED)	0x005C	R/W
MCPWM_DTO_RED_CFG_REG	Shadow register for rising edge delay (RED)	0x0060	R/W
MCPWM_CARRIER0_CFG_REG	Carrier enable and configuration	0x0064	R/W
MCPWM_FH0_CFG0_REG	Actions on PWM0A and PWM0B trip events	0x0068	R/W
MCPWM_FH0_CFG1_REG	Software triggers for fault handler actions	0x006C	R/W
MCPWM_FH0_STATUS_REG	Status of fault events	0x0070	RO
MCPWM Operator 1 Configuration and Status			
MCPWM_GEN1_STMP_CFG_REG	Transfer status and update method for time stamp registers A and B	0x0074	varies
MCPWM_GEN1_TSTMP_A_REG	Shadow register for register A	0x0078	R/W
MCPWM_GEN1_TSTMP_B_REG	Shadow register for register B	0x007C	R/W
MCPWM_GEN1_CFG0_REG	Fault event T0 and T1 handling	0x0080	R/W
MCPWM_GEN1_FORCE_REG	Permissives to force PWM1A and PWM1B outputs by software	0x0084	R/W
MCPWM_GEN1_A_REG	Actions triggered by events on PWM1A	0x0088	R/W
MCPWM_GEN1_B_REG	Actions triggered by events on PWM1B	0x008C	R/W
MCPWM_DT1_CFG_REG	Dead time type selection and configuration	0x0090	R/W
MCPWM_DT1_FED_CFG_REG	Shadow register for falling edge delay (FED)	0x0094	R/W
MCPWM_DT1_RED_CFG_REG	Shadow register for rising edge delay (RED)	0x0098	R/W
MCPWM_CARRIER1_CFG_REG	Carrier enable and configuration	0x009C	R/W
MCPWM_FH1_CFG0_REG	Actions on PWM1A and PWM1B trip events	0x00A0	R/W
MCPWM_FH1_CFG1_REG	Software triggers for fault handler actions	0x00A4	R/W
MCPWM_FH1_STATUS_REG	Status of fault events	0x00A8	RO
MCPWM Operator 2 Configuration and Status			
MCPWM_GEN2_STMP_CFG_REG	Transfer status and update method for time stamp registers A and B	0x00AC	varies
MCPWM_GEN2_TSTMP_A_REG	Shadow register for register A	0x00B0	R/W
MCPWM_GEN2_TSTMP_B_REG	Shadow register for register B	0x00B4	R/W
MCPWM_GEN2_CFG0_REG	Fault event T0 and T1 handling	0x00B8	R/W
MCPWM_GEN2_FORCE_REG	Permissives to force PWM2A and PWM2B outputs by software	0x00BC	R/W
MCPWM_GEN2_A_REG	Actions triggered by events on PWM2A	0x00C0	R/W
MCPWM_GEN2_B_REG	Actions triggered by events on PWM2B	0x00C4	R/W
MCPWM_DT2_CFG_REG	Dead time type selection and configuration	0x00C8	R/W
MCPWM_DT2_FED_CFG_REG	Shadow register for falling edge delay (FED)	0x00CC	R/W
MCPWM_DT2_RED_CFG_REG	Shadow register for rising edge delay (RED)	0x00D0	R/W
MCPWM_CARRIER2_CFG_REG	Carrier enable and configuration	0x00D4	R/W
MCPWM_FH2_CFG0_REG	Actions on PWM2A and PWM2B trip events	0x00D8	R/W
MCPWM_FH2_CFG1_REG	Software triggers for fault handler actions	0x00DC	R/W
MCPWM_FH2_STATUS_REG	Status of fault events	0x00E0	RO

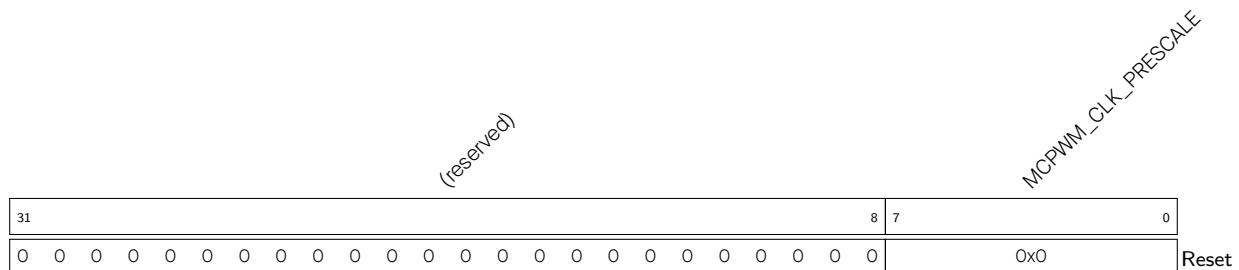
Name	Description	Address	Access
Fault Detection Configuration and Status			
MCPWM_FAULT_DETECT_REG	Fault detection configuration and status	0x00E4	varies
Capture Configuration and Status			
MCPWM_CAP_TIMER_CFG_REG	Configure capture timer	0x00E8	varies
MCPWM_CAP_TIMER_PHASE_REG	Phase for capture timer sync	0x00EC	R/W
MCPWM_CAP_CHO_CFG_REG	Capture channel 0 configuration and enable	0x00F0	varies
MCPWM_CAP_CH1_CFG_REG	Capture channel 1 configuration and enable	0x00F4	varies
MCPWM_CAP_CH2_CFG_REG	Capture channel 2 configuration and enable	0x00F8	varies
MCPWM_CAP_CHO_REG	ch0 capture value status register	0x00FC	RO
MCPWM_CAP_CH1_REG	ch1 capture value status register	0x0100	RO
MCPWM_CAP_CH2_REG	ch2 capture value status register	0x0104	RO
MCPWM_CAP_STATUS_REG	Edge of last capture trigger	0x0108	RO
Enable Update of Active Registers			
MCPWM_UPDATE_CFG_REG	Enable update	0x010C	R/W
Manage Interrupts			
MCPWM_INT_ENA_REG	Interrupt enable bits	0x0110	R/W
MCPWM_INT_RAW_REG	Raw interrupt status	0x0114	R/WTC /SS
MCPWM_INT_ST_REG	Masked interrupt status	0x0118	RO
MCPWM_INT_CLR_REG	Interrupt clear bits	0x011C	WT
MCPWM Event Enable Registers			
MCPWM_EVT_EN_REG	MCPWM event enable register	0x0120	R/W
MCPWM_EVT_EN2_REG	MCWM event enable register2	0x0128	R/W
MCPWM Task Enable Register			
MCPWM_TASK_EN_REG	MCPWM task enable register	0x0124	R/W
MCPWM Generator Configuration Registers			
MCPWM_OPO_TSTMP_E1_REG	Generator0 time stamp E1 value configuration register	0x012C	R/W
MCPWM_OPO_TSTMP_E2_REG	Generator0 time stamp E2 value configuration register	0x0130	R/W
MCPWM_OP1_TSTMP_E1_REG	Generator1 time stamp E1 value configuration register	0x0134	R/W
MCPWM_OP1_TSTMP_E2_REG	Generator1 time stamp E2 value configuration register	0x0138	R/W
MCPWM_OP2_TSTMP_E1_REG	Generator2 time stamp E1 value configuration register	0x013C	R/W
MCPWM_OP2_TSTMP_E2_REG	Generator2 time stamp E2 value configuration register	0x0140	R/W
MCPWM APB Configuration Register			
MCPWM_CLK_REG	MCPWM APB configuration register	0x0144	R/W
Version Register			
MCPWM_VERSION_REG	Version control register	0x0148	R/W

56.6 Registers

The addresses in this section are relative to Motor Control PWM base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

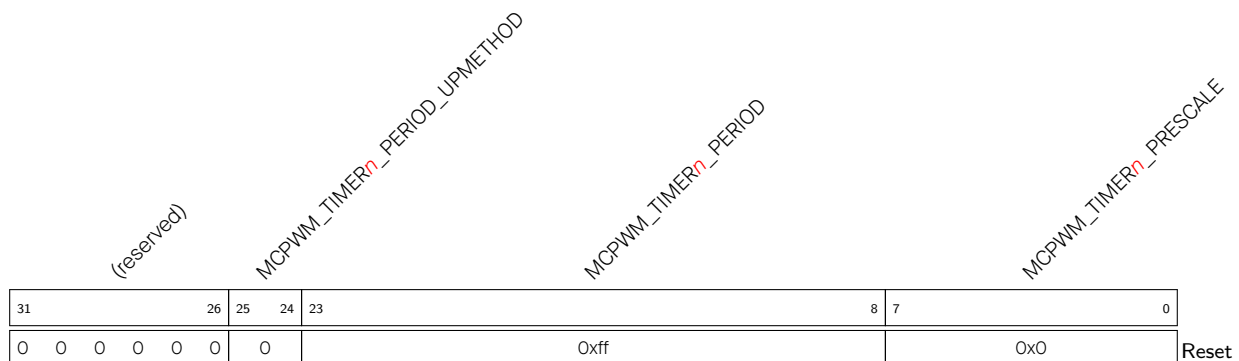
For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 56.1. MCPWM_CLK_CFG_REG (0x0000)



MCPWM_CLK_PRESCALE Configures the prescaler value of the clock, so that the period of $PWM_clk = 6.25ns * (PWM_CLK_PRESCALE + 1)$. (R/W)

Register 56.2. MCPWM_TIMER_n_CFG0_REG (*n*: 0-2) (0x0004+0x10n*)**



MCPWM_TIMER_n_PRESCALE Configures the prescaler value of timer_n, so that the period of $PTO_clk = \text{Period of } PWM_clk * (PWM_TIMER_n_PRESCALE + 1)$. (R/W)

MCPWM_TIMER_n_PERIOD Configures the period shadow of PWM timer_n. (R/W)

MCPWM_TIMER_n_PERIOD_UPMETHOD Configures the update method for active register of PWM timer_n period.

0: Immediate

1: TEZ

2: Sync

3: TEZ or sync

TEZ here and below means timer equals zero event.

(R/W)

Register 56.3. MCPWM_TIMERn_CFG1_REG(*n*: 0-2) (0x0008+0x10**n*)

(reserved)																MCPWM_TIMERn_MOD		MCPWM_TIMERn_START	
31															5	4	3	2	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0x0	0x0	Reset	

MCPWM_TIMERn_START Configures conditions to start/stop PWM timer*n*.

- 0: If PWM timer0 starts, then stops at TEZ
- 1: If timer0 starts, then stops at TEP
- 2: PWM timer0 starts and runs on
- 3: Timer0 starts and stops at the next TEZ
- 4: Timer0 starts and stops at the next TEP
- 5: Invalid. No effect
- 6: Invalid. No effect
- 7: Invalid. No effect

TEP here and below means the event that happens when the timer equals to period.
(R/W/SC)

MCPWM_TIMERn_MOD Configures the working mode of PWM timer*n*.

- 0: Freeze
- 1: Increase mode
- 2: Decrease mode
- 3: Up-down mode

(R/W)

Register 56.4. MCPWM_TIMER n _SYNC_REG (n : 0-2) (0x000C+0x10* n)

[illegible]

MCPWM_TIMER n _SYNCR_EN Configures whether to enable timer n reloading with phase on sync input event.

0: Disable

1: Enable

(R/W)

MCPWM_TIMER n _SYNC_SW Configures whether to trigger a software sync.

0: No effect

1: Trigger a software sync

(R/W)

MCPWM_TIMER n _SYNCO_SEL Configures PWM timer n sync out selection.

0: sync_in. The sync out will always generate when toggling the `MCPWM_TIMER0_SYNC_SW` bit.

1: TEZ

2: TEP

3: No effect

(R/W)

MCPWM_TIMER n _PHASE Configures the phase for timer n reload on sync event.

(R/W)

MCPWM_TIMER n _PHASE_DIRECTION Configures the PWM timer n 's direction when timer n mode is up-down mode.

0: Increase

1: Decrease

(R/W)

Register 56.5. MCPWM_TIMER n _STATUS_REG(n : 0-2) (0x0010+0x10* n)

Diagram illustrating the structure of the MCPWM_TIMERn register (n = 0, 1, 2, 3):

- Bit 31 to 17: (reserved)
- Bit 16 to 15: MCPWM_TIMERn_DIRECTION
- Bit 14 to 0: MCPWM_TIMERn_VALUE

The register is shown with a 'Reset' value of 0 for all bits.

MCPWM_TIMER n _VALUE Represents current PWM timer n counter value. (RO)

MCPWM_TIMER n _DIRECTION Represents current PWM timer n counter direction.

0: Increment

1: Decrement

(RO)

Register 56.6. MCPWM_TIMER_SYNCI_CFG_REG (0x0034)

Register 0x00000000: MCPWM_TIM00_SYNCSEL

Bit	Field
31-12	(reserved)
11-8	MCPWM_TIMER0_SYNCSEL
7-4	MCPWM_TIMER1_SYNCSEL
3-0	MCPWM_TIMER2_SYNCSEL
11-8	MCPWM_EXTERNAL_SYNCIO_INVERT

Reset: 00000000000000000000000000000000

MCPWM_TIMER n _SYNCISEL Configures sync input for PWM timer n .

1: PWM timer0 sync out

2: PWM timer1 sync out

3: PWM timer2 sync out

4: SYNC0 from GPIO matrix

5: SYNC1 from GPIO matrix

6: SYNC2 from GPIO matrix

Other values: no sync input selected

(R/W)

MCPWM_EXTERNAL_SYNCIn_INVERT Configures whether to invert SYNC_n from GPIO matrix.

0: Not invert

1: Invert

(R/W)

(reserved)

MCPWM_OPERATOR2_TIMERSEL
MCPWM_OPERATOR1_TIMERSEL
MCPWM_OPERATOR0_TIMERSEL

operator n .

(R/W)

Register 56.8. MCPWM_GEN n _STMP_CFG_REG (n : 0-2) (0x003C+0x38* n)

(reserved)										MCPWM_CMPR n _B_SHDW_FULL				MCPWM_CMPR n _A_SHDW_FULL				MCPWM_CMPR n _B_UPMETHOD				MCPWM_CMPR n _A_UPMETHOD			
31										10	9	8	7		4	3									
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

Reset

MCPWM_CMPR n _A_UPMETHOD Configures the update method for PWM generator n time stamp A's active register.

When all bits are set to 0: Immediately

When bit0 is set to 1: TEZ

When bit1 is set to 1: TEP

When bit2 is set to 1: Sync

When bit3 is set to 1: Disable the update

(R/W)

MCPWM_CMPR n _B_UPMETHOD Configures the update method for PWM generator n time stamp B's active register.

When all bits are set to 0: Immediately

When bit0 is set to 1: TEZ

When bit1 is set to 1: TEP

When bit2 is set to 1: Sync

When bit3 is set to 1: Disable the update

(R/W)

MCPWM_CMPR n _A_SHDW_FULL Configures whether the value in the shadow register is written to the corresponding active register at a specific time. This field is set and reset by hardware.

0: Write the latest value in the shadow register to A's active register

1: Write the value to the PWM generator n time stamp A's shadow register, and wait to be transferred to A's active register

(R/SC/WTC)

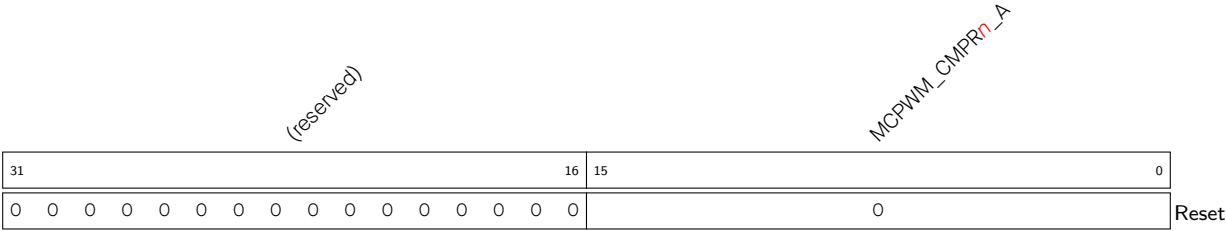
MCPWM_CMPR n _B_SHDW_FULL Configures whether the value in the shadow register is written to the corresponding active register at a specific time. This field is set and reset by hardware.

0: Write the latest value in the shadow register to B's active register

1: Write the value to the PWM generator n time stamp B's shadow register, and wait to be transferred to B's active register

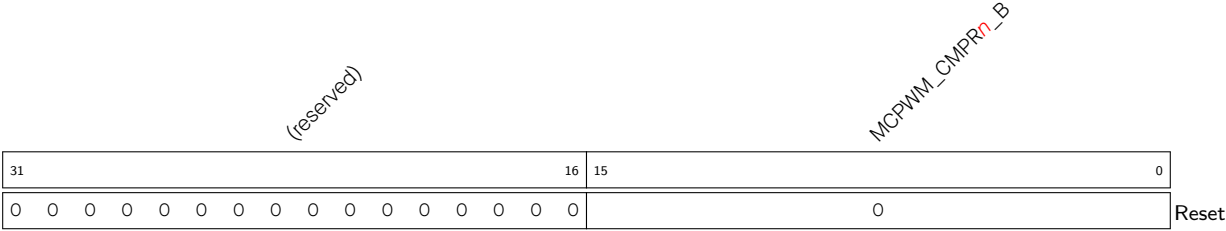
(R/SC/WTC)

Register 56.9. MCPWM_GENn_TSTMP_A_REG(*n*: 0-2) (0x0040+0x38**n*)



MCPWM_CMPRn_A Configures the value of PWM generator *n* time stamp A's shadow register.
(R/W)

Register 56.10. MCPWM_GENn_TSTMP_B_REG(*n*: 0-2) (0x0044+0x38**n*)



MCPWM_CMPRn_B Configures the value of PWM generator *n* time stamp B's shadow register.
(R/W)

Register 56.11. MCPWM_GEN n _CFG0_REG(n : 0-2) (0x0048+0x38* n)

31	10	9	7	6	4	3	0
(reserved)							
MCPWM_GEN_n_T1_SEL							
MCPWM_GEN_n_T0_SEL							
MCPWM_GEN_n_CFG_UPMETHOD							
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

MCPWM_GEN n _CFG_UPMETHOD Configures the update method for PWM generator n 's active register.

When all bits are set to 0: Immediately

When bit0 is set to 1: TEZ

When bit1 is set to 1: TEP

When bit2 is set to 1: Sync

When bit3 is set to 1: Disable the update

(R/W)

MCPWM_GEN n _TO_SEL Configures source selection for PWM generator n event_t0, take effect immediately.

0: fault_event0

```
1: fault event1
```

2: fault_event2

3: sync_taken

4: Invalid, selects nothing

(R/W)

MCPWM_GEN*n*_T1_SEL Configures source selection for PWM generator*n* event_t1, take effect immediately.

0: fault_event0

1: fault_event1

```
2: fault event2
```

3: sync_taken

4: Invalid, selects nothing

(R/W)

Register 56.12. MCPWM_GEN n _FORCE_REG(n : 0-2) (0x004C+0x38* n)

(reserved)																MCPWM_GEN _n _B_NCIFORCE_MODE					MCPWM_GEN _n _B_NCIFORCE					MCPWM_GEN _n _A_NCIFORCE_MODE					MCPWM_GEN _n _A_NCIFORCE					MCPWM_GEN _n _B_CNTUFORCE_MODE					MCPWM_GEN _n _A_CNTUFORCE_MODE					MCPWM_GEN _n _CNTUFORCE_UPMETHOD				
31																16	15	14	13	12	11	10	9	8	7	6	5	0																						
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0	0	0	0	0	0	0	0	0x20	Reset																								

MCPWM_GEN n _CNTUFORCE_UPMETHOD Configures the update method for continuous software force of PWM generator n .

When all bits are set to 0: Immediately

When bit0 is set to 1: TEZ

When bit1 is set to 1: TEP

When bit2 is set to 1: TEA

When bit3 is set to 1: TEB

When bit4 is set to 1: Sync

When bit5 is set to 1: Disable update

TEA/B means an event generated when the timer's value equals to that of register A/B.

(R/W)

MCPWM_GEN n _A_CNTUFORCE_MODE Configures the continuous software force mode for PWM n

A.

0: Disabled

1: Low

2: High

3: Disabled

(R/W)

MCPWM_GEN n _B_CNTUFORCE_MODE Configures the continuous software force mode for PWM n

B.

0: Disabled

1: Low

2: High

3: Disabled

(R/W)

MCPWM_GEN n _A_NCIFORCE Configures whether to trigger the non-continuous immediate software-force event for PWM n A.

0: Invalid

1: Trigger a force event

(R/W)

Continued on the next page...

Register 56.12. MCPWM_GEN n _FORCE_REG(n : 0-2) (0x004C+0x38* n)

Continued from the previous page...

MCPWM_GEN n _A_NCIFORCE_MODE Configures non-continuous immediate software force mode for PWM n A.

0: Disabled

1: Low

2: High

3: Disabled

(R/W)

MCPWM_GEN n _B_NCIFORCE Configures whether to trigger the non-continuous immediate software-force event for PWM n B.

0: Invalid

1: Trigger a force event

(R/W)

MCPWM_GEN n _B_NCIFORCE_MODE Configures non-continuous immediate software force mode for PWM n B.

0: Disabled

1: Low

2: High

3: Disabled

(R/W)

Register 56.13. MCPWM_GEN n _A_REG(n : 0-2) (0x0050+0x38* n)

(reserved)								MCPWM_GEN n _A_DT1		MCPWM_GEN n _A_DTO		MCPWM_GEN n _A_DTEB		MCPWM_GEN n _A_DTEA		MCPWM_GEN n _A_DTEP		MCPWM_GEN n _A_DTEZ		MCPWM_GEN n _A_UT1		MCPWM_GEN n _A_UTO		MCPWM_GEN n _A_UTEB		MCPWM_GEN n _A_UTEA		MCPWM_GEN n _A_UTEZ			
31							24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Reset

MCPWM_GEN n _A_UTEZ Configures action on PWM n A triggered by event TEZ when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _A_UTEZ Configures action on PWM n A triggered by event TEP when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _A_UTEA Configures action on PWM n A triggered by event TEA when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _A_UTEB Configures action on PWM n A triggered by event TEB when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

Continued on the next page...

Register 56.13. MCPWM_GEN n _A_REG(n : 0-2) (0x0050+0x38* n)

Continued from the previous page...

MCPWM_GEN n _A_UT0 Configures action on PWM n A triggered by event_t0 when timer increasing.

- 0: No change
- 1: Low
- 2: High
- 3: Toggle
- (R/W)

MCPWM_GEN n _A_UT1 Configures action on PWM n A triggered by event_t1 when timer increasing.

- 0: No change
- 1: Low
- 2: High
- 3: Toggle
- (R/W)

MCPWM_GEN n _A_DTEZ Configures action on PWM n A triggered by event TEZ when timer decreasing.

- 0: No change
- 1: Low
- 2: High
- 3: Toggle
- (R/W)

MCPWM_GEN n _A_DTEP Configures action on PWM n A triggered by event TEP when timer decreasing.

- 0: No change
- 1: Low
- 2: High
- 3: Toggle
- (R/W)

MCPWM_GEN n _A_DTEA Configures action on PWM n A triggered by event TEA when timer decreasing.

- 0: No change
- 1: Low
- 2: High
- 3: Toggle
- (R/W)

Continued on the next page...

Register 56.13. MCPWM_GEN n _A_REG(n : 0-2) (0x0050+0x38* n)

Continued from the previous page...

MCPWM_GEN n _A_DTEB Configures action on PWM n A triggered by event TEB when timer decreasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _A_DTO Configures action on PWM n A triggered by event_t0 when timer decreasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _A_DT1 Configures action on PWM n A triggered by event_t1 when timer decreasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

Register 56.14. MCPWM_GEN n _B_REG(n : 0-2) (0x0054+0x38* n)

(reserved)								MCPWM_GEN _n _B_DT1		MCPWM_GEN _n _B_DTO		MCPWM_GEN _n _B_DTEB		MCPWM_GEN _n _B_DTEA		MCPWM_GEN _n _B_DTEP		MCPWM_GEN _n _B_DTEZ		MCPWM_GEN _n _B_UT1		MCPWM_GEN _n _B_UTO		MCPWM_GEN _n _B_UTEA		MCPWM_GEN _n _B_UTEZ						
31								24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_GEN n _B_UTEZ Configures action on PWM n B triggered by event TEZ when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _B_UTEZ Configures action on PWM n B triggered by event TEP when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _B_UTEA Configures action on PWM n B triggered by event TEA when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _B_UTEB Configures action on PWM n B triggered by event TEB when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _B_UTO Configures action on PWM n B triggered by event_t0 when timer increasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

Continued on the next page...

Register 56.14. MCPWM_GEN n _B_REG(n : 0-2) (0x0054+0x38* n)

Continued from the previous page...

MCPWM_GEN n _B_UT1 Configures action on PWM n B triggered by event_t1 when timer increasing.

- 0: No change
 - 1: Low
 - 2: High
 - 3: Toggle
- (R/W)

MCPWM_GEN n _B_DTEZ Configures action on PWM n B triggered by event TEZ when timer decreasing.

- 0: No change
 - 1: Low
 - 2: High
 - 3: Toggle
- (R/W)

MCPWM_GEN n _B_DTEP Configures action on PWM n B triggered by event TEP when timer decreasing.

- 0: No change
 - 1: Low
 - 2: High
 - 3: Toggle
- (R/W)

MCPWM_GEN n _B_DTEA Configures action on PWM n B triggered by event TEA when timer decreasing.

- 0: No change
 - 1: Low
 - 2: High
 - 3: Toggle
- (R/W)

MCPWM_GEN n _B_DTEB Configures action on PWM n B triggered by event TEB when timer decreasing.

- 0: No change
 - 1: Low
 - 2: High
 - 3: Toggle
- (R/W)

Continued on the next page...

Register 56.14. MCPWM_GEN n _B_REG(n : 0-2) (0x0054+0x38* n)

Continued from the previous page...

MCPWM_GEN n _B_DTO Configures action on PWM n B triggered by event_t0 when timer decreasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

MCPWM_GEN n _B_DT1 Configures action on PWM n B triggered by event_t1 when timer decreasing.

0: No change

1: Low

2: High

3: Toggle

(R/W)

Register 56.15. MCPWM_DT n _CFG_REG(n : 0-2) (0x0058+0x38* n)

(reserved)																MCPWM_DB _n _CLK_SEL MCPWM_DB _n _B_OUTBYPASS MCPWM_DB _n _A_OUTBYPASS MCPWM_DB _n _FED_OUTINVERT MCPWM_DB _n _RED_OUTINVERT MCPWM_DB _n _FED_INSEL MCPWM_DB _n _RED_INSEL MCPWM_DB _n _A_OUTSWAP MCPWM_DB _n _DEB_MODE MCPWM_DB _n _RED_UPMETHOD MCPWM_DB _n _FED_UPMETHOD																
31																18	17	16	15	14	13	12	11	10	9	8	7	4		3	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		0		Reset									

MCPWM_DBn_FED_UPMETHOD Configures the update method for FED (Falling edge delay) active register.

0: Immediate

Bit0 is set to 1: TEZ

Bit1 is set to 1: TEP

Bit2 is set to 1: Sync

Bit3 is set to 1: Disable the update

(R/W)

MCPWM_DBn_RED_UPMETHOD Configures the update method for RED (Rising edge delay) active register.

0: Immediate

Bit0 is set to 1: TEZ

Bit1 is set to 1: TEP

Bit2 is set to 1: Sync

Bit3 is set to 1: Disable the update

(R/W)

MCPWM_DB n _DEB_MODE Configures the S8 switch in Table 56.3-5.

0: FED/RED take effect on different paths separately

1: FED/RED take effect on B path, A out is in bypass or dulpB mode

(R/W)

MCPWM_DBn_A_OUTSWAP Configures the S6 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DBn_B_OUTSWAP Configures the S7 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DBn_RED_INSEL Configures the S4 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DBn_FED_INSEL Configures the S5 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DBn_RED_OUTINVERT Configures the S2 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

Continued on the next page...

Register 56.15. MCPWM_DT n _CFG_REG(n : 0-2) (0x0058+0x38* n)

Continued from the previous page...

MCPWM_DB n _FED_OUTINVERT Configures the S3 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DB n _A_OUTBYPASS Configures the S1 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DB n _B_OUTBYPASS Configures the S0 switch in Table 56.3-5. For typical configurations, please refer to Table 56.3-6. (R/W)

MCPWM_DB n _CLK_SEL Configures dead time generator n clock selection.

0: PWM_CLK

1: PT_CLK

(R/W)

Register 56.16. MCPWM_DT n _FED_CFG_REG(n : 0-2) (0x005C+0x38* n)

(reserved)																MCPWM_DB _n _FED															
31															16	15															0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0															
Reset																															

MCPWM_DB n _FED Configures the shadow register for FED. (R/W)

Register 56.17. MCPWM_DT n _RED_CFG_REG(n : 0-2) (0x0060+0x38* n)

(reserved)																MCPWM_DB _n _RED																
31																0																
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0																Reset

MCPWM_DB n _RED Configures the shadow register for RED. (R/W)

Register 56.18. MCPWM_CARRIER n _CFG_REG(n : 0-2) (0x0064+0x38* n)

(reserved)																MCPWM_CHOPPER n _IN_INVERT		MCPWM_CHOPPER n _OUT_INVERT		MCPWM_CHOPPER n _OSHTWTH		MCPWM_CHOPPER n _DUTY		MCPWM_CHOPPER n _PRESCALE		MCPWM_CHOPPER n _EN		
31																14	13	12	11	8		7	5		4	1		0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0	0	0		0		0		0		Reset		

MCPWM_CHOPPER n _EN Configures whether to enable carrier n .

0: Bypassed

1: Enabled

(R/W)

MCPWM_CHOPPER n _PRESCALE Configures the prescale value of PWM carrier n clock (PC_clk), so that period of PC_clk = period of PWM_clk * (PWM_CARRIER n _PRESCALE + 1). (R/W)

MCPWM_CHOPPER n _DUTY Configures carrier duty. Duty = PWM_CARRIER n _DUTY / 8. (R/W)

MCPWM_CHOPPER n _OSHTWTH Configures width of the first pulse. Measurement unit: Periods of the carrier. (R/W)

MCPWM_CHOPPER n _OUT_INVERT Configures whether to invert the output of PWM n A and PWM n B for this submodule.

0: Normal

1: Invert

(R/W)

MCPWM_CHOPPER n _IN_INVERT Configures whether to invert the input of PWM n A and PWM n B for this submodule.

0: Normal

1: Invert

(R/W)

Register 56.19. MCPWM_FH_n_CFG0_REG(*n*: 0-2) (0x0068+0x38**n*)

[illegible]

MCPWM_TZ_n_SW_CBC Configures whether to enable software force cycle-by-cycle mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZ*n*_F2_CBC Configures whether event_f2 will trigger cycle-by-cycle mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZ*n*_F1_CBC Configures whether event_f1 will trigger cycle-by-cycle mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZ_n_FO_CBC Configures whether event_f0 will trigger cycle-by-cycle mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZ*n*_SW_OST Configures whether to enable software force one-shot mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZn_F2_OST Configures whether event_f2 will trigger one-shot mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZn_F1_OST Configures whether event_f1 will trigger one-shot mode action.

0: Disable

1: Enable

(R/W)

MCPWM_TZn_Fn_OST Configures whether event_f0 will trigger one-shot mode action.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 56.19. MCPWM_FH_n_CFG0_REG(*n*: 0-2) (0x0068+0x38n*)**

Continued from the previous page...

MCPWM_TZ_n_A_CBC_D Configures cycle-by-cycle mode action on PWM_n A when fault event occurs and timer is decreasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_n_A_CBC_U Configures cycle-by-cycle mode action on PWM_n A when fault event occurs and timer is increasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_n_A_OST_D Configures one-shot mode action on PWM_n A when fault event occurs and timer is decreasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_n_A_OST_U Configures one-shot mode action on PWM_n A when fault event occurs and timer is increasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_n_B_CBC_D Configures cycle-by-cycle mode action on PWM_n B when fault event occurs and timer is decreasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

Continued on the next page...

Register 56.19. MCPWM_FH_{*n*}_CFG0_REG(*n*: 0-2) (0x0068+0x38n*)**

Continued from the previous page...

MCPWM_TZ_{*n*}_B_CBC_U Configures cycle-by-cycle mode action on PWM_{*n*} B when fault event occurs and timer is increasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_{*n*}_B_OST_D Configures one-shot mode action on PWM_{*n*} B when fault event occurs and timer is decreasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

MCPWM_TZ_{*n*}_B_OST_U Configures one-shot mode action on PWM_{*n*} B when fault event occurs and timer is increasing.

- 0: Do nothing
 - 1: Force low
 - 2: Force high
 - 3: Toggle
- (R/W)

Register 56.20. MCPWM_FH_n_CFG1_REG(*n*: 0-2) (0x006C+0x38**n*)

(reserved)																MCPWM_TZ _n _FORCE_OST					
																MCPWM_TZ _n _FORCE_CBC					
																MCPWM_TZ _n _CBCPULSE					
																MCPWM_TZ _n _CLR_OST					
31																5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
																0	0	0	0	0	0
Reset																					

MCPWM_TZ_n_CLR_OST Configures whether to generate a one-shot mode action clear by software.

0: No effect

1: Triggers a clear for ongoing one-shot mode action by software

(R/W)

MCPWM_TZ_n_CBCPULSE Configures the refresh moment selection of cycle-by-cycle mode action.

0: Select nothing, will not refresh

Bit0 is set to 1: TEZ

Bit1 is set to 1: TEP

(R/W)

MCPWM_TZ_n_FORCE_CBC Configures whether to generate a software cycle-by-cycle mode action.

0: No effect

1: Triggers a cycle-by-cycle mode action by software

(R/W)

MCPWM_TZ_n_FORCE_OST Configures whether to generate a software one-shot mode action.

0: No effect

1: Triggers a one-shot mode action by software

(R/W)

Register 56.21. MCPWM_FH_{*n*}_STATUS_REG(*n*: 0-2) (0x0070+0x38**n*)

(reserved)																								MCPWM_TZ _{<i>n</i>} _OST_ON		MCPWM_TZ _{<i>n</i>} _CBC_ON	
31																							2	1	0		
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_TZ_{*n*}_CBC_ON Represents whether an cycle-by-cycle mode action is on going.

0: No action

1: Ongoing

(RO)

MCPWM_TZ_{*n*}_OST_ON Represents whether a one-shot mode action is ongoing.

0: No action

1: Ongoing

(RO)

Register 56.22. MCPWM_FAULT_DETECT_REG (0x00E4)

(reserved)																						MCPWM_EVENT_F2		MCPWM_EVENT_F1		MCPWM_EVENT_F0					
31																						9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset	

MCPWM_F_{*n*}_EN Configures whether to enable event_*fn* generation.

0: Disable

1: Enable

(R/W)

MCPWM_F_{*n*}_POLE Configures event_*fn* trigger polarity on FAULT_{*n*} source from GPIO matrix.

0: Level low

1: Level high

(R/W)

MCPWM_EVENT_F_{*n*} Represents whether an event_*fn* is ongoing. This field is set and reset by hardware.

0: No action

1: Ongoing

(RO)

Register 56.23. MCPWM_CAP_TIMER_CFG_REG (0x00E8)

(reserved)																										MCPWM_CAP_SYNC_SW		MCPWM_CAP_SYNCI_SEL		MCPWM_CAP_SYNCI_EN		MCPWM_CAP_TIMER_EN	
31																										6	5	4		2		1	0
0 0																										0	0		0		0	Reset	

Reset

MCPWM_CAP_TIMER_EN Configures whether to enable capture timer increment incrementing under APB_CLK.

0: Disable

1: Enable

(R/W)

MCPWM_CAP_SYNCI_EN Configures whether to enable capture timer sync.

0: Disable

1: Enable

(R/W)

MCPWM_CAP_SYNCI_SEL Configures the selection of capture module sync input.

0: None

1: Timer0 sync_out

2: Timer1 sync_out

3: Timer2 sync_out

4: SYNC0 from GPIO matrix

5: SYNC1 from GPIO matrix

6: SYNC2 from GPIO matrix

7: None

(R/W)

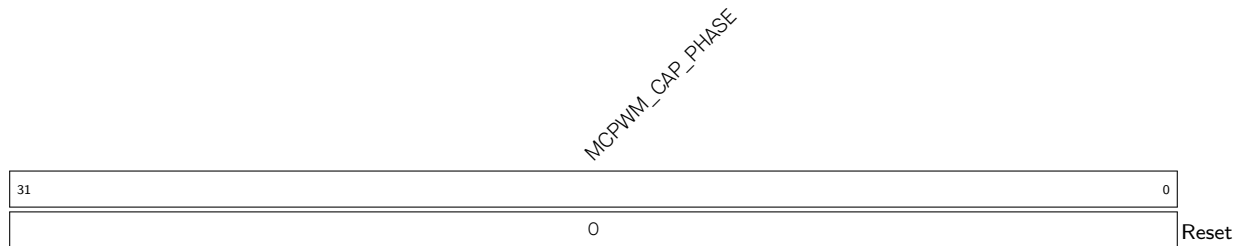
MCPWM_CAP_SYNC_SW When [MCPWM_CAP_SYNCI_EN](#) is set to 1, configures whether to trigger a capture timer sync so that capture timer is loaded with value in phase register.

0: No effect

1: Trigger a capture timer sync

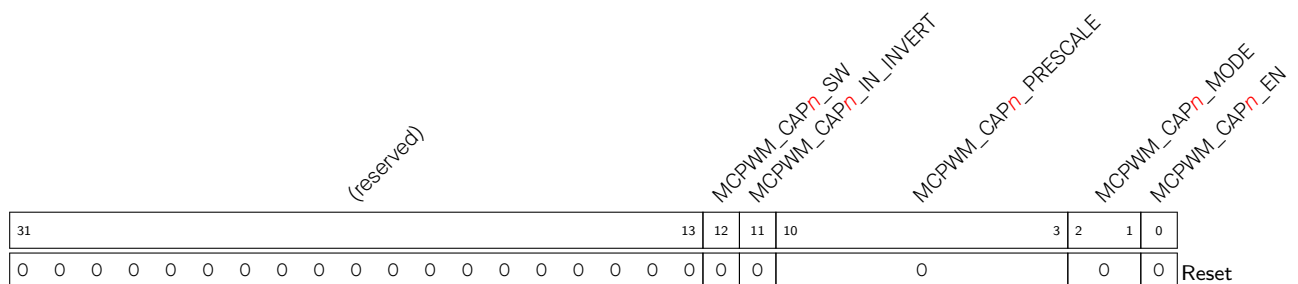
(WT)

Register 56.24. MCPWM_CAP_TIMER_PHASE_REG (0x00EC)



MCPWM_CAP_PHASE Configures phase value for capture timer sync operation.
(R/W)

Register 56.25. MCPWM_CAP_CH n _CFG_REG (n : 0-2) (0x00F0+0x4* n)



MCPWM_CAP n _EN Configures whether to enable capture on channel n .

0: Disable

1: Enable

(R/W)

MCPWM_CAPn_MODE Configures the edge of capture on channel 0 after prescaling.
 When bit0 is set to 1: enable capture on the falling edge.
 When bit1 is set to 1: enable capture on the rising edge.
 (R/W)

MCPWM_CAP*n*_PRESCALE Configures the prescale value on the rising edge of CAPO. Prescale value = PWM_CAPO_PRESCALE + 1. (R/W)

MCPWM_CAP n _IN_INVERT Configures whether to invert CAP n from GPIO matrix before prescale.
 0: Normal
 1: Invert
 (R/W)

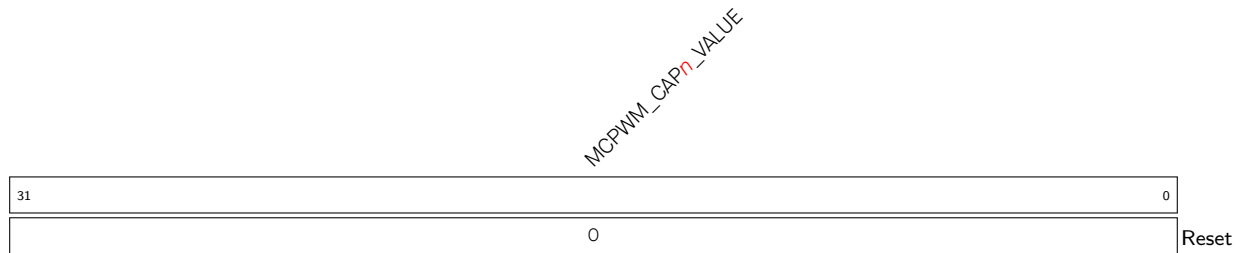
MCPWM_CAP n _SW Configures whether to trigger a software-forced capture on channel 0.

0: Not trigger

1: Trigger

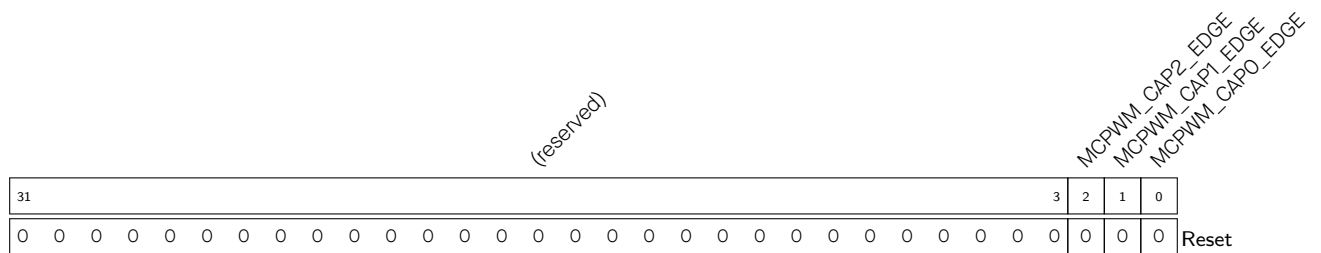
(WT)

Register 56.26. MCPWM_CAP_CH n _REG (n : 0-2) (0x00FC+0x4* n)



MCPWM_CAP_n_VALUE Represents value of last capture on CAP_n. (RO)

Register 56.27. MCPWM_CAP_STATUS_REG (0x0108)



MCPWM_CAP n _EDGE Represents the edge of the last capture trigger on channel n .

0: Rising edge

1: Falling edge

(RO)

Register 56.28. MCPWM_UPDATE_CFG_REG (0x010C)

(reserved)																												MCPWM_OP2_FORCE_UP								MCPWM_OP2_UP_EN								MCPWM_OP1_FORCE_UP								MCPWM_OP1_UP_EN								MCPWM_OPO_FORCE_UP								MCPWM_OPO_UP_EN								MCPWM_GLOBAL_FORCE_UP								MCPWM_GLOBAL_UP_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																						
31																												8		7	6	5	4	3	2	1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																					
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0</

MCPWM_GLOBAL_UP_EN Configures whether to globally update all active registers.

0: No effect

1: Update all active registers globally

(R/W)

MCPWM_GLOBAL_FORCE_UP Configures whether to trigger a forced update of all active registers globally.

0: No effect

1: Trigger a forced update

(R/W)

MCPWM_OPO_UP_EN Configures whether to update active registers in PWM operator 0 when [MCPWM_GLOBAL_UP_EN](#) is set to 1.

0: No effect

1: Update active registers in PWM operator 0

(R/W)

MCPWM_OPO_FORCE_UP Configures whether to trigger a forced update of active registers in PWM operator 0.

0: No effect

1: Trigger a forced update

(R/W)

MCPWM_OP1_UP_EN Configures whether to update active registers in PWM operator 1 when [MCPWM_GLOBAL_UP_EN](#) is set to 1.

0: No effect

1: Update active registers in PWM operator 1

(R/W)

Continued on the next page...

Register 56.28. MCPWM_UPDATE_CFG_REG (0x010C)

Continued from the previous page...

MCPWM_OP1_FORCE_UP Configures whether to trigger a forced update of active registers in PWM operator 1.

0: No effect

1: Trigger a forced update

(R/W)

MCPWM_OP2_UP_EN Configures whether to update active registers in PWM operator 2 when [MCPWM_GLOBAL_UP_EN](#) is set to 1.

0: No effect

1: Update active registers in PWM operator 2

(R/W)

MCPWM_OP2_FORCE_UP Configures whether to trigger a forced update of active registers in PWM operator 2.

0: No effect

1: Trigger a forced update

(R/W)

Register 56.29. MCPWM_INT_ENA_REG (0x0110)

(reserved) MCPWM_CAP2_INT_ENA MCPWM_CAP1_INT_ENA MCPWM_CAP0_INT_ENA MCPWM_TZ2_INT_ENA MCPWM_TZ1_OST_INT_ENA MCPWM_TZ0_OST_INT_ENA MCPWM_TZ1_OST_INT_ENA																															
---	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

MCPWM_TIMER0_STOP_INT_ENA Write 1 to enable [MCPWM_TIMER0_STOP_INT](#). (R/W)

MCPWM_TIMER1_STOP_INT_ENA Write 1 to enable [MCPWM_TIMER1_STOP_INT](#). (R/W)

MCPWM_TIMER2_STOP_INT_ENA Write 1 to enable [MCPWM_TIMER2_STOP_INT](#). (R/W)

MCPWM_TIMER0_TEZ_INT_ENA Write 1 to enable [MCPWM_TIMER0_TEZ_INT](#). (R/W)

MCPWM_TIMER1_TEZ_INT_ENA Write 1 to enable [MCPWM_TIMER1_TEZ_INT](#). (R/W)

MCPWM_TIMER2_TEZ_INT_ENA Write 1 to enable [MCPWM_TIMER2_TEZ_INT](#). (R/W)

MCPWM_TIMER0_TEP_INT_ENA Write 1 to enable [MCPWM_TIMER0_TEP_INT](#). (R/W)

MCPWM_TIMER1_TEP_INT_ENA Write 1 to enable [MCPWM_TIMER1_TEP_INT](#). (R/W)

MCPWM_TIMER2_TEP_INT_ENA Write 1 to enable [MCPWM_TIMER2_TEP_INT](#). (R/W)

MCPWM_FAULT0_INT_ENA Write 1 to enable [MCPWM_FAULT0_INT](#). (R/W)

MCPWM_FAULT1_INT_ENA Write 1 to enable [MCPWM_FAULT1_INT](#). (R/W)

MCPWM_FAULT2_INT_ENA Write 1 to enable [MCPWM_FAULT2_INT](#). (R/W)

MCPWM_FAULT0_CLR_INT_ENA Write 1 to enable [MCPWM_FAULT0_CLR_INT](#). (R/W)

MCPWM_FAULT1_CLR_INT_ENA Write 1 to enable [MCPWM_FAULT1_CLR_INT](#). (R/W)

MCPWM_FAULT2_CLR_INT_ENA Write 1 to enable [MCPWM_FAULT2_CLR_INT](#). (R/W)

MCPWM_CMPRO_TEA_INT_ENA Write 1 to enable [MCPWM_CMPRO_TEA_INT](#). (R/W)

MCPWM_CMPR1_TEA_INT_ENA Write 1 to enable [MCPWM_CMPR1_TEA_INT](#). (R/W)

MCPWM_CMPR2_TEA_INT_ENA Write 1 to enable [MCPWM_CMPR2_TEA_INT](#). (R/W)

MCPWM_CMPRO_TEB_INT_ENA Write 1 to enable [MCPWM_CMPRO_TEB_INT](#). (R/W)

MCPWM_CMPR1_TEB_INT_ENA Write 1 to enable [MCPWM_CMPR1_TEB_INT](#). (R/W)

MCPWM_CMPR2_TEB_INT_ENA Write 1 to enable [MCPWM_CMPR2_TEB_INT](#). (R/W)

MCPWM_TZO_CBC_INT_ENA Write 1 to enable [MCPWM_TZO_CBC_INT](#). (R/W)

Continued on the next page...

Register 56.29. MCPWM_INT_ENA_REG (0x0110)

Continued from the previous page...

MCPWM_TZ1_CBC_INT_ENA Write 1 to enable [MCPWM_TZ1_CBC_INT](#). (R/W)

MCPWM_TZ2_CBC_INT_ENA Write 1 to enable [MCPWM_TZ2_CBC_INT](#). (R/W)

MCPWM_TZ0_OST_INT_ENA Write 1 to enable [MCPWM_TZ0_OST_INT](#). (R/W)

MCPWM_TZ1_OST_INT_ENA Write 1 to enable [MCPWM_TZ1_OST_INT](#). (R/W)

MCPWM_TZ2_OST_INT_ENA Write 1 to enable [MCPWM_TZ2_OST_INT](#). (R/W)

MCPWM_CAPO_INT_ENA Write 1 to enable [MCPWM_CAPO_INT](#). (R/W)

MCPWM_CAP1_INT_ENA Write 1 to enable [MCPWM_CAP1_INT](#). (R/W)

MCPWM_CAP2_INT_ENA Write 1 to enable [MCPWM_CAP2_INT](#). (R/W)

Register 56.30. MCPWM_INT_RAW_REG (0x0114)

(reserved) MCPWM_CAP2_INT_RAW MCPWM_CAP1_INT_RAW MCPWM_CAP0_INT_RAW MCPWM_TZ2_OST_INT_RAW MCPWM_TZ1_OST_INT_RAW MCPWM_TZ0_OST_INT_RAW MCPWM_TZ2_CBC_INT_RAW MCPWM_TZ1_CBC_INT_RAW MCPWM_TZ0_CBC_INT_RAW MCPWM_CMPR2_INT_RAW MCPWM_CMPR1_TEB_INT_RAW MCPWM_CMPR0_TEB_INT_RAW MCPWM_CMPR2_TEA_INT_RAW MCPWM_CMPR1_TEA_INT_RAW MCPWM_FAULT2_CLR_INT_RAW MCPWM_FAULT1_CLR_INT_RAW MCPWM_FAULT0_CLR_INT_RAW MCPWM_TIMER2_TEP_INT_RAW MCPWM_TIMER1_TEP_INT_RAW MCPWM_TIMER0_TEP_INT_RAW MCPWM_TIMER2_TEZ_INT_RAW MCPWM_TIMER1_TEZ_INT_RAW MCPWM_TIMER0_TEZ_INT_RAW MCPWM_TIMER2_STOP_INT_RAW MCPWM_TIMER1_STOP_INT_RAW MCPWM_TIMER0_STOP_INT_RAW																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_TIMER0_STOP_INT_RAW Represents the raw status of [MCPWM_TIMER0_STOP_INT](#).
(R/WTC/SS)

MCPWM_TIMER1_STOP_INT_RAW Represents the raw status of [MCPWM_TIMER1_STOP_INT](#).
(R/WTC/SS)

MCPWM_TIMER2_STOP_INT_RAW Represents the raw status of [MCPWM_TIMER2_STOP_INT](#).
(R/WTC/SS)

MCPWM_TIMER0_TEZ_INT_RAW Represents the raw status of [MCPWM_TIMER0_TEZ_INT](#).
(R/WTC/SS)

MCPWM_TIMER1_TEZ_INT_RAW Represents the raw status of [MCPWM_TIMER1_TEZ_INT](#).
(R/WTC/SS)

MCPWM_TIMER2_TEZ_INT_RAW Represents the raw status of [MCPWM_TIMER2_TEZ_INT](#).
(R/WTC/SS)

MCPWM_TIMER0_TEP_INT_RAW Represents the raw status of [MCPWM_TIMER0_TEP_INT](#).
(R/WTC/SS)

MCPWM_TIMER1_TEP_INT_RAW Represents the raw status of [MCPWM_TIMER1_TEP_INT](#).
(R/WTC/SS)

MCPWM_TIMER2_TEP_INT_RAW Represents the raw status of [MCPWM_TIMER2_TEP_INT](#).
(R/WTC/SS)

MCPWM_FAULT0_INT_RAW Represents the raw status of [MCPWM_FAULT0_INT](#). (R/WTC/SS)

MCPWM_FAULT1_INT_RAW Represents the raw status of [MCPWM_FAULT1_INT](#). (R/WTC/SS)

MCPWM_FAULT2_INT_RAW Represents the raw status of [MCPWM_FAULT2_INT](#). (R/WTC/SS)

MCPWM_FAULT0_CLR_INT_RAW Represents the raw status of [MCPWM_FAULT0_CLR_INT](#).
(R/WTC/SS)

MCPWM_FAULT1_CLR_INT_RAW Represents the raw status of [MCPWM_FAULT1_CLR_INT](#).
(R/WTC/SS)

Continued on the next page...

Register 56.30. MCPWM_INT_RAW_REG (0x0114)

Continued from the previous page...

MCPWM_FAULT2_CLR_INT_RAW Represents the raw status of [MCPWM_FAULT2_CLR_INT](#).
(R/WTC/SS)

MCPWM_CMPRO_TEA_INT_RAW Represents the raw status of [MCPWM_CMPRO_TEA_INT](#).
(R/WTC/SS)

MCPWM_CMPR1_TEA_INT_RAW Represents the raw status of [MCPWM_CMPR1_TEA_INT](#).
(R/WTC/SS)

MCPWM_CMPR2_TEA_INT_RAW Represents the raw status of [MCPWM_CMPR2_TEA_INT](#).
(R/WTC/SS)

MCPWM_CMPRO_TEB_INT_RAW Represents the raw status of [MCPWM_CMPRO_TEB_INT](#).
(R/WTC/SS)

MCPWM_CMPR1_TEB_INT_RAW Represents the raw status of [MCPWM_CMPR1_TEB_INT](#).
(R/WTC/SS)

MCPWM_CMPR2_TEB_INT_RAW Represents the raw status of [MCPWM_CMPR2_TEB_INT](#).
(R/WTC/SS)

MCPWM_TZO_CBC_INT_RAW Represents the raw status of [MCPWM_TZO_CBC_INT](#). (R/WTC/SS)

MCPWM_TZ1_CBC_INT_RAW Represents the raw status of [MCPWM_TZ1_CBC_INT](#). (R/WTC/SS)

MCPWM_TZ2_CBC_INT_RAW Represents the raw status of [MCPWM_TZ2_CBC_INT](#). (R/WTC/SS)

MCPWM_TZO_OST_INT_RAW Represents the raw status of [MCPWM_TZO_OST_INT](#). (R/WTC/SS)

MCPWM_TZ1_OST_INT_RAW Represents the raw status of [MCPWM_TZ1_OST_INT](#). (R/WTC/SS)

MCPWM_TZ2_OST_INT_RAW Represents the raw status of [MCPWM_TZ2_OST_INT](#). (R/WTC/SS)

MCPWM_CAPO_INT_RAW Represents the raw status of [MCPWM_CAPO_INT](#). (R/WTC/SS)

MCPWM_CAP1_INT_RAW Represents the raw status of [MCPWM_CAP1_INT](#). (R/WTC/SS)

MCPWM_CAP2_INT_RAW Represents the raw status of [MCPWM_CAP2_INT](#). (R/WTC/SS)

Register 56.31. MCPWM_INT_ST_REG (0x0118)

(reserved) MCPWM_CAP2_INT_ST MCPWM_CAP1_INT_ST MCPWM_CAO_INT_ST MCPWM_TZ2_OST_INT_ST MCPWM_TZ1_OST_INT_ST MCPWM_TZ0_OST_INT_ST MCPWM_TZ2_CBC_INT_ST MCPWM_TZ1_CBC_INT_ST MCPWM_TZ0_CBC_INT_ST MCPWM_CMPR2_TEB_INT_ST MCPWM_CMPR1_TEB_INT_ST MCPWM_CMPR2_TEA_INT_ST MCPWM_CMPR1_TEA_INT_ST MCPWM_FAULT2_CLR_INT_ST MCPWM_FAULT1_CLR_INT_ST MCPWM_FAULT2_INT_ST MCPWM_FAULT1_INT_ST MCPWM_TIMER2_TEP_INT_ST MCPWM_TIMER1_TEP_INT_ST MCPWM_TIMER0_TEP_INT_ST MCPWM_TIMER2_TEZ_INT_ST MCPWM_TIMER1_TEZ_INT_ST MCPWM_TIMER0_TEZ_INT_ST MCPWM_TIMER2_STOP_INT_ST MCPWM_TIMER1_STOP_INT_ST MCPWM_TIMER0_STOP_INT_ST																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

Reset

MCPWM_TIMER0_STOP_INT_ST Represents the masked status of [MCPWM_TIMER0_STOP_INT](#). (RO)

MCPWM_TIMER1_STOP_INT_ST Represents the masked status of [MCPWM_TIMER1_STOP_INT](#). (RO)

MCPWM_TIMER2_STOP_INT_ST Represents the masked status of [MCPWM_TIMER2_STOP_INT](#). (RO)

MCPWM_TIMER0_TEZ_INT_ST Represents the masked status of [MCPWM_TIMER0_TEZ_INT](#). (RO)

MCPWM_TIMER1_TEZ_INT_ST Represents the masked status of [MCPWM_TIMER1_TEZ_INT](#). (RO)

MCPWM_TIMER2_TEZ_INT_ST Represents the masked status of [MCPWM_TIMER2_TEZ_INT](#). (RO)

MCPWM_TIMER0_TEP_INT_ST Represents the masked status of [MCPWM_TIMER0_TEP_INT](#). (RO)

MCPWM_TIMER1_TEP_INT_ST Represents the masked status of [MCPWM_TIMER1_TEP_INT](#). (RO)

MCPWM_TIMER2_TEP_INT_ST Represents the masked status of [MCPWM_TIMER2_TEP_INT](#). (RO)

MCPWM_FAULT0_INT_ST Represents the masked status of [MCPWM_FAULT0_INT](#). (RO)

MCPWM_FAULT1_INT_ST Represents the masked status of [MCPWM_FAULT1_INT](#). (RO)

MCPWM_FAULT2_INT_ST Represents the masked status of [MCPWM_FAULT2_INT](#). (RO)

MCPWM_FAULT0_CLR_INT_ST Represents the masked status of [MCPWM_FAULT0_CLR_INT](#). (RO)

MCPWM_FAULT1_CLR_INT_ST Represents the masked status of [MCPWM_FAULT1_CLR_INT](#). (RO)

Continued on the next page...

Register 56.31. MCPWM_INT_ST_REG (0x0118)

Continued from the previous page...

MCPWM_FAULT2_CLR_INT_ST Represents the masked status of [MCPWM_FAULT2_CLR_INT](#). (RO)

MCPWM_CMPRO_TEA_INT_ST Represents the masked status of [MCPWM_CMPRO_TEA_INT](#). (RO)

MCPWM_CMPR1_TEA_INT_ST Represents the masked status of [MCPWM_CMPR1_TEA_INT](#). (RO)

MCPWM_CMPR2_TEA_INT_ST Represents the masked status of [MCPWM_CMPR2_TEA_INT](#). (RO)

MCPWM_CMPRO_TEB_INT_ST Represents the masked status of [MCPWM_CMPRO_TEB_INT](#). (RO)

MCPWM_CMPR1_TEB_INT_ST Represents the masked status of [MCPWM_CMPR1_TEB_INT](#). (RO)

MCPWM_CMPR2_TEB_INT_ST Represents the masked status of [MCPWM_CMPR2_TEB_INT](#). (RO)

MCPWM_TZO_CBC_INT_ST Represents the masked status of [MCPWM_TZO_CBC_INT_ST](#). (RO)

MCPWM_TZ1_CBC_INT_ST Represents the masked status of [MCPWM_TZ1_CBC_INT_ST](#). (RO)

MCPWM_TZ2_CBC_INT_ST Represents the masked status of [MCPWM_TZ2_CBC_INT_ST](#). (RO)

MCPWM_TZO_OST_INT_ST Represents the masked status of [MCPWM_TZO_OST_INT](#). (RO)

MCPWM_TZ1_OST_INT_ST Represents the masked status of [MCPWM_TZ1_OST_INT](#). (RO)

MCPWM_TZ2_OST_INT_ST Represents the masked status of [MCPWM_TZ2_OST_INT](#). (RO)

MCPWM_CAPO_INT_ST Represents the masked status of [MCPWM_CAPO_INT](#). (RO)

MCPWM_CAP1_INT_ST Represents the masked status of [MCPWM_CAP1_INT](#). (RO)

MCPWM_CAP2_INT_ST Represents the masked status of [MCPWM_CAP2_INT](#). (RO)

Register 56.32. MCPWM_INT_CLR_REG (0x011C)

(reserved) MCPWM_CAP2_INT_CLR MCPWM_CAP1_INT_CLR MCPWM_CAPO_INT_CLR MCPWM_TZ2_OST_INT_CLR MCPWM_TZ1_OST_INT_CLR MCPWM_TZ2_OST_INT_CLR MCPWM_TZ1_OST_INT_CLR MCPWM_TZ2_CBC_INT_CLR MCPWM_TZ1_CBC_INT_CLR MCPWM_TZ0_CBC_INT_CLR MCPWM_CMPR2_TEB_INT_CLR MCPWM_CMPR1_TEB_INT_CLR MCPWM_CMPRO_TEA_INT_CLR MCPWM_FAULT2_INT_CLR MCPWM_FAULT1_INT_CLR MCPWM_FAULT0_INT_CLR MCPWM_TIMER2_TEP_INT_CLR MCPWM_TIMER1_TEP_INT_CLR MCPWM_TIMER2_TEP_INT_CLR MCPWM_TIMER1_TEP_INT_CLR MCPWM_TIMER2_TEZ_INT_CLR MCPWM_TIMER1_TEZ_INT_CLR MCPWM_TIMER2_STOP_INT_CLR MCPWM_TIMER1_STOP_INT_CLR MCPWM_TIMER0_STOP_INT_CLR																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_TIMER0_STOP_INT_CLR Write 1 to clear [MCPWM_TIMER0_STOP_INT](#). (WT)

MCPWM_TIMER1_STOP_INT_CLR Write 1 to clear [MCPWM_TIMER1_STOP_INT](#). (WT)

MCPWM_TIMER2_STOP_INT_CLR Write 1 to clear [MCPWM_TIMER2_STOP_INT](#). (WT)

MCPWM_TIMER0_TEZ_INT_CLR Write 1 to clear [MCPWM_TIMER0_TEZ_INT](#). (WT)

MCPWM_TIMER1_TEZ_INT_CLR Write 1 to clear [MCPWM_TIMER1_TEZ_INT](#). (WT)

MCPWM_TIMER2_TEZ_INT_CLR Write 1 to clear [MCPWM_TIMER2_TEZ_INT](#). (WT)

MCPWM_TIMER0_TEP_INT_CLR Write 1 to clear [MCPWM_TIMER0_TEP_INT](#). (WT)

MCPWM_TIMER1_TEP_INT_CLR Write 1 to clear [MCPWM_TIMER1_TEP_INT](#). (WT)

MCPWM_TIMER2_TEP_INT_CLR Write 1 to clear [MCPWM_TIMER2_TEP_INT](#). (WT)

MCPWM_FAULT0_INT_CLR Write 1 to clear [MCPWM_FAULT0_INT](#). (WT)

MCPWM_FAULT1_INT_CLR Write 1 to clear [MCPWM_FAULT1_INT](#). (WT)

MCPWM_FAULT2_INT_CLR Write 1 to clear [MCPWM_FAULT2_INT](#). (WT)

MCPWM_FAULT0_CLR_INT_CLR Write 1 to clear [MCPWM_FAULT0_CLR_INT](#). (WT)

MCPWM_FAULT1_CLR_INT_CLR Write 1 to clear [MCPWM_FAULT1_CLR_INT](#). (WT)

MCPWM_FAULT2_CLR_INT_CLR Write 1 to clear [MCPWM_FAULT2_CLR_INT](#). (WT)

MCPWM_CMPRO_TEA_INT_CLR Write 1 to clear [MCPWM_CMPRO_TEA_INT](#). (WT)

MCPWM_CMPR1_TEA_INT_CLR Write 1 to clear [MCPWM_CMPR1_TEA_INT](#). (WT)

MCPWM_CMPR2_TEA_INT_CLR Write 1 to clear [MCPWM_CMPR2_TEA_INT](#). (WT)

MCPWM_CMPRO_TEB_INT_CLR Write 1 to clear [MCPWM_CMPRO_TEB_INT](#). (WT)

MCPWM_CMPR1_TEB_INT_CLR Write 1 to clear [MCPWM_CMPR1_TEB_INT](#). (WT)

MCPWM_CMPR2_TEB_INT_CLR Write 1 to clear [MCPWM_CMPR2_TEB_INT](#). (WT)

MCPWM_TZ0_CBC_INT_CLR Write 1 to clear [MCPWM_TZ0_CBC_INT](#). (WT)

Continued on the next page...

Register 56.32. MCPWM_INT_CLR_REG (0x011C)

Continued from the previous page...

MCPWM_TZ1_CBC_INT_CLR Write 1 to clear [MCPWM_TZ1_CBC_INT](#). (WT)

MCPWM_TZ2_CBC_INT_CLR Write 1 to clear [MCPWM_TZ2_CBC_INT](#). (WT)

MCPWM_TZ0_OST_INT_CLR Write 1 to clear [MCPWM_TZ0_OST_INT](#). (WT)

MCPWM_TZ1_OST_INT_CLR Write 1 to clear [MCPWM_TZ1_OST_INT](#). (WT)

MCPWM_TZ2_OST_INT_CLR Write 1 to clear [MCPWM_TZ2_OST_INT](#). (WT)

MCPWM_CAPO_INT_CLR Write 1 to clear [MCPWM_CAPO_INT](#). (WT)

MCPWM_CAP1_INT_CLR Write 1 to clear [MCPWM_CAP1_INT](#). (WT)

MCPWM_CAP2_INT_CLR Write 1 to clear [MCPWM_CAP2_INT](#). (WT)

Register 56.33. MCPWM_EVT_EN_REG (0x0120)

(reserved) MCPWM_EVT_CAP2_EN MCPWM_EVT_CAP1_EN MCPWM_EVT_CAP0_EN MCPWM_EVT_TZ2_EN MCPWM_EVT_TZ1_EN MCPWM_EVT_TZ0_EN MCPWM_EVT_TZ2_EN MCPWM_EVT_TZ1_EN MCPWM_EVT_TZ0_EN MCPWM_EVT_TZ2_EN MCPWM_EVT_TZ1_EN MCPWM_EVT_TZ0_EN MCPWM_EVT_F2_CBC_EN MCPWM_EVT_F1_CBC_EN MCPWM_EVT_F1_CLR_EN MCPWM_EVT_F0_CLR_EN MCPWM_EVT_F2_EN MCPWM_EVT_F1_EN MCPWM_EVT_F0_EN MCPWM_EVT_OP2_TEB_EN MCPWM_EVT_OP1_TEB_EN MCPWM_EVT_OPO_TEB_EN MCPWM_EVT_OP2_TEA_EN MCPWM_EVT_OPO_TEA_EN MCPWM_EVT_TIMER2_EN MCPWM_EVT_TIMER1_TEP_EN MCPWM_EVT_TIMER0_TEP_EN MCPWM_EVT_TIMER2_TEP_EN MCPWM_EVT_TIMER1_TEA_EN MCPWM_EVT_TIMER0_TEA_EN MCPWM_EVT_TIMER2_STOP_EN MCPWM_EVT_TIMER1_STOP_EN MCPWM_EVT_TIMER0_STOP_EN																																
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset

MCPWM_EVT_TIMER0_STOP_EN Configures whether to enable timer0 stop event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER1_STOP_EN Configures whether to enable timer1 stop event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER2_STOP_EN Configures whether to enable timer2 stop event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER0_TEZ_EN Configures whether to enable timer0 equal zero event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER1_TEZ_EN Configures whether to enable timer1 equal zero event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER2_TEZ_EN Configures whether to enable timer2 equal zero event generation.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 56.33. MCPWM_EVT_EN_REG (0x0120)

Continued from the previous page...

MCPWM_EVT_TIMER0_TEP_EN Configures whether to enable timer0 equal period event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER1_TEP_EN Configures whether to enable timer1 equal period event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TIMER2_TEP_EN Configures whether to enable timer2 equal period event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_OPO_TEA_EN Configures whether to enable PWM generator0 timer equal A event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_OP1_TEA_EN Configures whether to enable PWM generator1 timer equal A event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_OP2_TEA_EN Configures whether to enable PWM generator2 timer equal A event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_OPO_TEB_EN Configures whether to enable PWM generator0 timer equal B event generation.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 56.33. MCPWM_EVT_EN_REG (0x0120)

Continued from the previous page...

MCPWM_EVT_OP1_TEB_EN Configures whether to enable PWM generator1 timer equal B event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_OP2_TEB_EN Configures whether to enable PWM generator2 timer equal B event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F0_EN Configures whether to enable FAULT0 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F1_EN Configures whether to enable FAULT1 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F2_EN Configures whether to enable FAULT2 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F0_CLR_EN Configures whether to enable FAULT0 clear event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F1_CLR_EN Configures whether to enable FAULT1 clear event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_F2_CLR_EN Configures whether to enable FAULT2 clear event generation.

0: Disable

1: Enable

(R/W)

Continued on the next page...

Register 56.33. MCPWM_EVT_EN_REG (0x0120)

Continued from the previous page...

MCPWM_EVT_TZO_CBC_EN Configures whether to enable cycle by cycle trip0 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TZ1_CBC_EN Configures whether to enable cycle by cycle trip1 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TZ2_CBC_EN Configures whether to enable cycle by cycle trip2 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TZO_OST_EN Configures whether to enable one shot trip0 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TZ1_OST_EN Configures whether to enable one shot trip1 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_TZ2_OST_EN Configures whether to enable one shot trip2 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_CAPO_EN Configures whether to enable capture0 event generation.

0: Disable

1: Enable

(R/W)

MCPWM_EVT_CAP1_EN Configures whether to enable capture1 event generation.

0: Disable

1: Enable

(R/W)

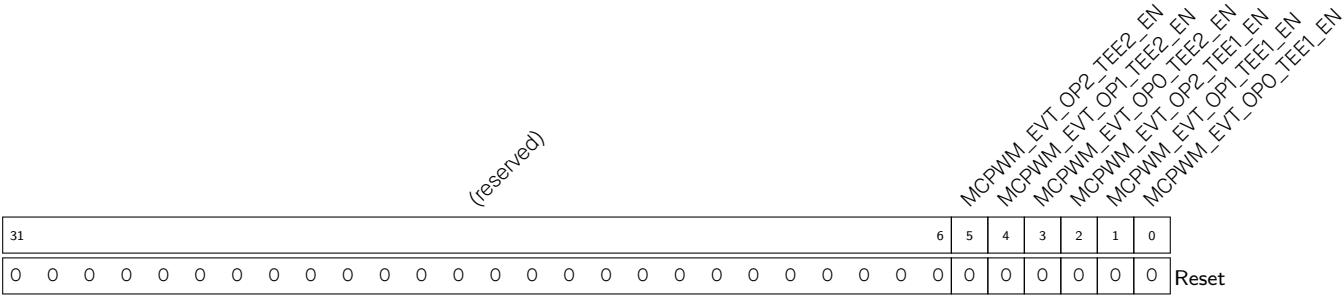
MCPWM_EVT_CAP2_EN Configures whether to enable capture2 event generation.

0: Disable

1: Enable

(R/W)

Register 56.34. MCPWM_EVT_EN2_REG (0x0128)



MCPWM_EVT_OPn_TEE1_EN Configures whether to generate the MCPWM_EVT_OPn_TEE1 event when the PWM generatorn timer equals OPn_TSTMP_E1_REG.

0: Not generate

1: Generate

(R/W)

MCPWM_EVT_OPn_TEE2_EN Configures whether to generate the MCPWM_EVT_OPn_TEE2 event when the PWM generatorn timer equals OPn_TSTMP_E2_REG.

0: Not generate

1: Generate

(R/W)

Register 56.35. MCPWM_TASK_EN_REG (0x0124)

(reserved)												MCPWM_TASK_CAP2_EN MCPWM_TASK_CAP1_EN MCPWM_TASK_CAP0_EN MCPWM_TASK_CLR2_EN MCPWM_TASK_CLR1_EN MCPWM_TASK_CLR0_EN MCPWM_TASK_TZ2_EN MCPWM_TASK_TZ1_EN MCPWM_TASK_TZ0_EN MCPWM_TASK_TIMER2_EN MCPWM_TASK_TIMER1_EN MCPWM_TASK_TIMER0_EN MCPWM_TASK_TIMER2_PERIOD_UP_EN MCPWM_TASK_TIMER1_PERIOD_UP_EN MCPWM_TASK_TIMER0_PERIOD_UP_EN MCPWM_TASK_TIMER2_SYNC_EN MCPWM_TASK_TIMER1_SYNC_EN MCPWM_TASK_TIMER0_SYNC_EN MCPWM_TASK_GEN_STOP_EN MCPWM_TASK_CMPR2_B_UP_EN MCPWM_TASK_CMPR1_B_UP_EN MCPWM_TASK_CMPR2_A_UP_EN MCPWM_TASK_CMPR1_A_UP_EN MCPWM_TASK_CMPR0_A_UP_EN																						
31												22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	Reset			

MCPWM_TASK_CMPRO_A_UP_EN Configures whether to receive update task of PWM generator0 timer stamp A's shadow register.
 0: No effect
 1: Receive
 (R/W)

MCPWM_TASK_CMPR1_A_UP_EN Configures whether to receive update task of PWM generator1 timer stamp A's shadow register.
 0: No effect
 1: Receive
 (R/W)

MCPWM_TASK_CMPR2_A_UP_EN Configures whether to receive update task of PWM generator2 timer stamp A's shadow register.
 0: No effect
 1: Receive
 (R/W)

MCPWM_TASK_CMPRO_B_UP_EN Configures whether to receive update task of PWM generator0 timer stamp B's shadow register.
 0: No effect
 1: Receive
 (R/W)

MCPWM_TASK_CMPR1_B_UP_EN Configures whether to receive update task of PWM generator1 timer stamp B's shadow register.
 0: No effect
 1: Receive
 (R/W)

MCPWM_TASK_CMPR2_B_UP_EN Configures whether to receive update task of PWM generator2 timer stamp B's shadow register.
 0: No effect
 1: Receive
 (R/W)

Continued on the next page...

Register 56.35. MCPWM_TASK_EN_REG (0x0124)

Continued from the previous page...

MCPWM_TASK_GEN_STOP_EN Configures whether to receive all PWM generate stop task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER0_SYNC_EN Configures whether to receive timer0 sync task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER1_SYNC_EN Configures whether to receive timer1 sync task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER2_SYNC_EN Configures whether to receive timer2 sync task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER0_PERIOD_UP_EN Configures whether to receive timer0 period update task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER1_PERIOD_UP_EN Configures whether to receive timer1 period update task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TIMER2_PERIOD_UP_EN Configures whether to receive timer2 period update task.

0: No effect

1: Receive

(R/W)

Continued on the next page...

Register 56.35. MCPWM_TASK_EN_REG (0x0124)

Continued from the previous page...

MCPWM_TASK_TZ0_OST_EN Configures whether to receive one shot trip0 task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TZ1_OST_EN Configures whether to receive one shot trip1 task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_TZ2_OST_EN Configures whether to receive one shot trip2 task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CLRO_OST_EN Configures whether to receive one shot trip0 clear task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CLR1_OST_EN Configures whether to receive one shot trip1 clear task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CLR2_OST_EN Configures whether to receive one shot trip2 clear task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CAPO_EN Configures whether to receive capture0 task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CAP1_EN Configures whether to receive capture1 task.

0: No effect

1: Receive

(R/W)

MCPWM_TASK_CAP2_EN Configures whether to receive capture2 task.

0: No effect

1: Receive

(R/W)

Register 56.36. MCPWM_OP n _TSTMP_E1_REG (n : 0-2) (0x012C+0x8* n)

(reserved)																MCPWM_OP _n _TSTMP_E1																
31															16	15																0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0																
																																Reset

MCPWM_OP n _TSTMP_E1 Configures the time stamp E1 value of the generator n . (R/W)

Register 56.37. MCPWM_OP n _TSTMP_E2_REG(n : 0-2) (0x0130+0x8* n)

(reserved)																MCPWM_OP _n _TSTMP_E2															
31																0															
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0																0															
Reset																															

MCPWM_OP n _TSTMP_E2 Configures the time stamp E2 value of the generator n . (R/W)

Register 56.38. MCPWM_CLK_REG (0x0144)

(reserved)																															MCPWM_CLK_EN																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																								
31																															1	0																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																																							
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	

MCPWM_CLK_EN Configures whether to force open register clock gate.

0: Open the clock gate only when application writes registers

1: Force open the clock gate for register

(R/W)

Register 56.39. MCPWM_VERSION_REG (0x0148)

(reserved)				MCPWM_DATE																									
31	28	27	0																										
0	0	0	0	0x2212290																									Reset

MCPWM_DATE Version control register. (R/W)